



Documentazione Del Software

DietiEstates25

Angelo Paoletta Alfredo Volpe

Marzo 2025



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II

Indice

1	Requisiti Del Software	4
1.1	Requisiti Funzionali	4
1.2	Requisiti non funzionali	4
1.3	Requisiti di dominio	5
1.3.1	Requisiti di dominio funzionali	5
1.3.2	Requisiti di dominio non funzionali	5
1.4	Use case Diagram	6
1.5	Tabella dei Casi D'Uso	7
1.6	Descrizione degli attori	7
1.7	Descrizione dei casi d'uso	7
1.8	Tabelle di Cockburn	9
1.8.1	Creazione di account per Agente Immobiliare	9
1.8.2	Visualizzare inserzioni	10
1.9	Mock-up	12
1.9.1	Creazione agente immobiliare	12
1.9.2	Creazione inserzione	13
1.10	Personas	14
1.10.1	Utenti generici	14
1.10.2	Agenti immobiliari	15
1.10.3	Amministratori	15
1.11	Glossario	16
2	Documento di Design del Sistema	17
2.1	Descrizione dell'architettura proposta	17
2.1.1	Perché un back-end monolitico?	17
2.1.2	Livello di Presentazione (Front-end)	18
2.1.3	Livello di Logica di Business (Back-end)	19
2.1.4	Livello di Persistenza (Database)	20
2.2	Tecnologie usate	21
2.2.1	JavaFX	21
2.2.2	Spring	21
2.2.3	PostgreSQL	22
2.2.4	Amazon Web Services	22
2.3	Schema di persistenza dei dati	23
2.3.1	Entità	23
2.3.2	Associazioni	24
2.4	Scelte di design dell'interfaccia utente	25
2.4.1	Scelta del font	25
2.4.2	Scelta dei Colori	26
2.4.3	Esempio di principi di Gestalt usati	27
2.5	Diagramma delle classi di design	28
2.5.1	Front-end	28
2.5.2	Back-end	29
2.5.3	Shared-Library	30
2.6	Diagrammi di sequenza di design	31
2.6.1	Caso d'uso: registrazione di un agente	31
2.6.2	Caso d'uso: creazione inserzione	33
3	Processo di sviluppo	37
3.1	Codice sorgente, pom, Dockerfile e docker-compose	37
3.2	Statistiche di GitHub	37
3.3	Report di qualità del codice	38

4	Testing e valutazione dell'usabilità	39
4.1	Piano di testing	39
4.1.1	Metodo <code>updateAnnuncioStats</code>	39
4.1.2	Metodo <code>registerAgenzia</code>	42
4.1.3	Descrizione dei test effettuati	48
4.1.4	Test su <code>AnnuncioService</code>	48
4.1.5	Test su <code>AgenziaService</code>	48
4.1.6	Utilizzo di Mockito	48
4.1.7	Utilizzo dei Test Parametrizzati	48
4.2	Valutazione dell'usabilità	49
4.2.1	Expert review	49
4.2.2	Inspections	50
4.2.3	Risultati del form	50
4.2.4	Valutazione interna delle risposte	52

1 Requisiti Del Software

1.1 Requisiti Funzionali

I requisiti funzionali del software non sono altro che i casi d'uso identificati al paragrafo 1.7

1.2 Requisiti non funzionali

- **Architettura del sistema:**

Deve essere un sistema distribuito con tre componenti principali:

- **Front-end:** Interfaccia utente che comunica con il back-end esclusivamente tramite API e deve essere sostituibile senza modifiche al back-end.
- **Back-end:** Gestisce la logica di business, accessibile tramite API REST. Deve essere indipendente dal front-end e supportare tecnologie di containerizzazione come Docker.
- **Database:** È completamente separato dal back-end e gestisce i dati del sistema.

- **Sicurezza e gestione delle credenziali:**

Le credenziali degli utenti devono essere salvate in modo sicuro.

- **Compatibilità tecnologica:**

È obbligatorio l'uso di linguaggi di programmazione Object-Oriented e che supportino il testing automatico.

- **Indipendenza dai provider esterni:**

La logica applicativa e la persistenza dei dati non devono dipendere esclusivamente da servizi esterni.

- **Registrazione:**

La registrazione è possibile solo se l'email rispetta la struttura standard (esempio@email.com) e la password è lunga almeno 8 caratteri, ha almeno una lettera maiuscola e un numero.

- **Reportistica conforme alle pratiche di agenzia:**

I report generati devono seguire formati standard riconosciuti per la gestione interna delle agenzie con formati PDF e CSV.

- **Creazione di account:**

L'amministratore per gestire gli account deve essere autenticato.

1.3 Requisiti di dominio

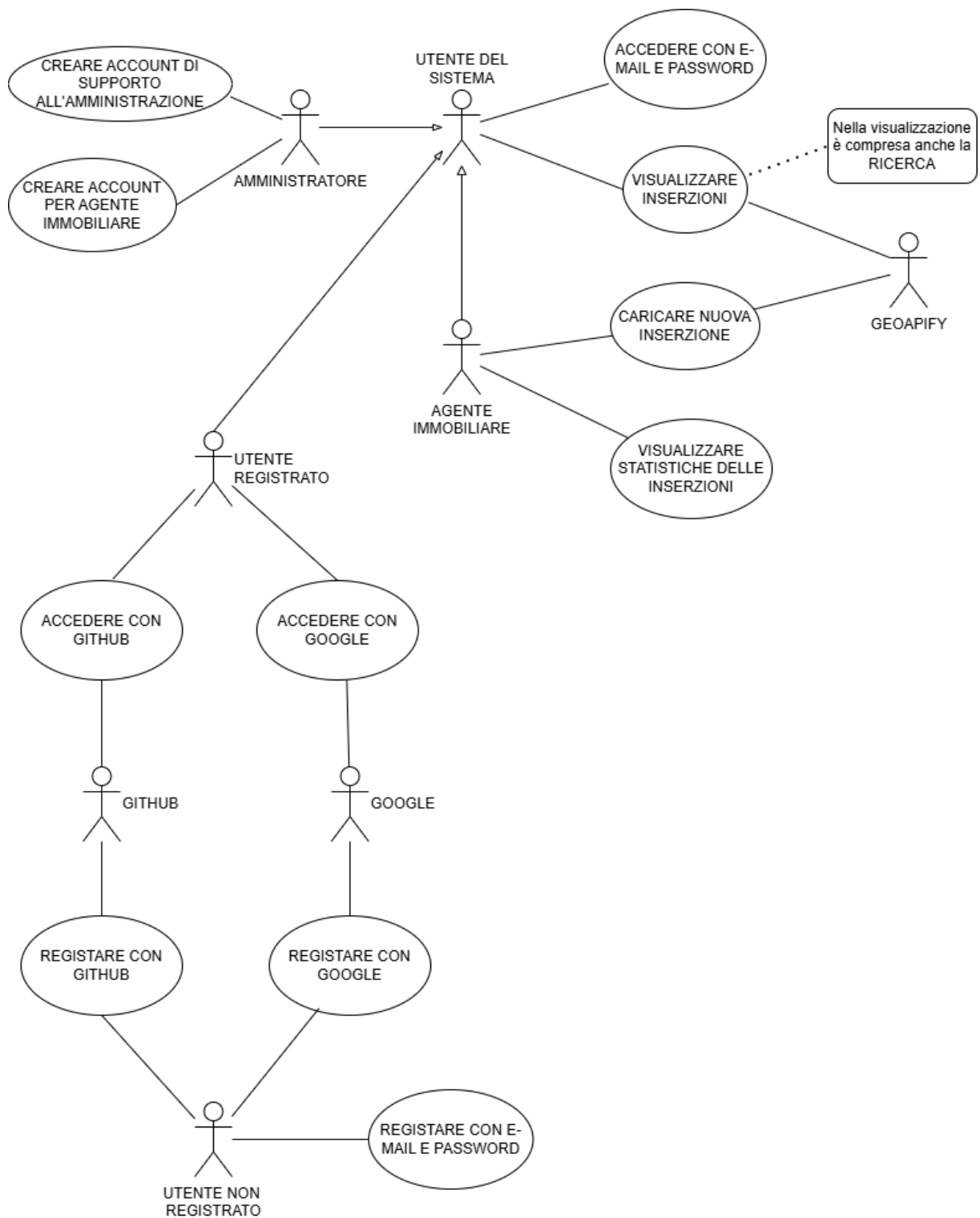
1.3.1 Requisiti di dominio funzionali

- **Gestione delle inserzioni:**
Un agente immobiliare autenticato può caricare nuovi immobili con dettagli specifici, categorizzando l'inserzione in "vendita" o "affitto" e molti altri filtri.
- **Ricerca dettagliata di immobili:**
Un utente del sistema può effettuare una ricerca basata su parametri multipli, inclusa la geolocalizzazione.
- **Verifica dei servizi nelle vicinanze:**
Controllo automatico sulle inserzioni caricate tramite Geoapify per vicinanza a scuole, parchi o trasporti pubblici.
- **Dashboard per gli agenti immobiliari:**
Gli agenti immobiliari autenticati possono visualizzare e scaricare statistiche e report delle proprie inserzioni.

1.3.2 Requisiti di dominio non funzionali

- **Standard di localizzazione geografica:**
L'uso della mappa interattiva (OpenLayers Map) per la localizzazione deve essere conforme agli standard richiesti dal settore immobiliare.
- **Integrazione di fonti esterne per dati contestuali:**
L'utilizzo di Geoapify per dati ambientali e demografici (es. per vicinanza a scuole) devono aderire agli standard richiesti per essere utilizzabili a fini decisionali.

1.4 Use case Diagram



1.5 Tabella dei Casi D'Uso

Attore principale	Casi d'uso
Utente Del Sistema	- Accedere con E-Mail e password - Visualizzare inserzioni
Amministratore	- Creare account di supporto all'amministrazione - Creare account per agente immobiliare
Agente Immobiliare	- Caricare nuova inserzione - Visualizzare statistiche delle inserzioni
Utente Registrato	- Accedere con Google - Accedere con Github
Utente Non Registrato	- Registrare con E-Mail e password - Registrare con Google - Registrare con Github

Tabella 1: Tabella con attori e casi d'uso

1.6 Descrizione degli attori

In questo paragrafo verranno descritti gli attori elencati nella tabella dei casi d'uso.

- **Utente Del Sistema:** tutti gli utenti che hanno accesso al sistema e che sono dotati di credenziali per accedervi.
- **Amministratore:** gestiscono gli account sul software della propria agenzia immobiliare.
- **Agente Immobiliare:** sono gli addetti alla vendita e all'affitto degli immobili.
- **Utente Registrato:** sono i potenziali acquirenti degli immobili che hanno effettuato la registrazione al software.
- **Utente Non Registrato:** sono utenti che non hanno effettuato una registrazione, avendo un accesso limitato al software.

1.7 Descrizione dei casi d'uso

- **Accedere con E-Mail e password:**
il sistema permette a un utente registrato di accedere con le proprie credenziali.
- **Visualizzare inserzioni:**
il sistema permette a tutti gli utenti autenticati di effettuare una ricerca e di visualizzare le inserzioni disponibili.
- **Creare account di supporto all'amministrazione:**
il sistema permette agli amministratori di creare altri account per il supporto all'amministrazione della propria agenzia immobiliare.
- **Creare Account per agente immobiliare:**
il sistema permette agli amministratori di creare account per gli agenti immobiliari della propria agenzia.
- **Caricare nuova inserzione:**
il sistema permette agli agenti immobiliari di caricare nuove inserzioni, inserendo svariati dettagli, foto, posizione e tipologia dell'immobile; inoltre devono specificare se è un affitto o una vendita.
- **Visualizzare statistiche delle inserzioni:**
gli agenti immobiliari hanno accesso a una dashboard con statistiche sugli immobili; inoltre possono esportarli in formato PDF o CSV.
- **Accedere con Google:**
gli utenti registrati tramite Google possono effettuarci l'accesso.

- **Accedere con Github:**
gli utenti registrati tramite Github possono effettuarci l'accesso.
- **Registrare con E-Mail e password:**
gli utenti non registrati possono effettuare la registrazione al software tramite E-Mail e password.
- **Registrare con Google:**
gli utenti non registrati possono effettuare la registrazione al software tramite Google.
- **Registrare con Github:**
gli utenti non registrati possono effettuare la registrazione al software tramite Github.

1.8 Tabelle di Cockburn

1.8.1 Creazione di account per Agente Immobiliare

USE CASE #1	Creare account per agente immobiliare		
Goal in Context	L'amministratore vuole creare un account per un agente immobiliare		
Preconditions	L'amministratore deve aver effettuato il caso d'uso "Accedere con e-mail e password"		
Success End Condition	Il sistema tiene traccia della creazione dell'account		
Failed End Condition	Il sistema non tiene traccia della creazione dell'account, ma del fallimento		
Primary Actor	Amministratore		
Trigger	L'amministratore clicca il pulsante "Creare account per un agente immobiliare" in M1		
Main scenario	Step n.	Amministratore	Sistema
	1	Clicca "Creare account per agente immobiliare" in M1	
	2		Mostra M2
	3	Compila tutti i dati e clicca "Registra"	
	4		Mostra M3
Extension A: L'amministratore decide di voler annullare l'operazione di creazione del account	3a	Clicca "Torna indietro"	
	4a		Mostra M1 per 3 secondi e torna a step 1 del Main scenario
Extension B: L'amministratore inserisce una data di nascita di una persona non maggiorenne	3b		Mostra ME1 per 3 secondi
	4b		Mostra M2 e torna a step 3 del Main scenario
Extension C: L'amministratore inserisce un'e-mail già esistente	4c		Mostra ME2 per 3 secondi
	5c		Mostra M2 e torna a step 3 del Main scenario
Extension D: Errore da parte del client	4d		Mostra ME3 per 3 secondi
	5d		Mostra M2 e torna a step 3 del Main scenario
Extension E: Errore da parte del Server	4e		Mostra ME4 per 3 secondi
	5e		Mostra M2 e torna a step 3 del Main scenario
Extension F: Errore generico	4f		Mostra ME5 per 3 secondi
	5f		Mostra M2 e torna a step 3 del Main scenario

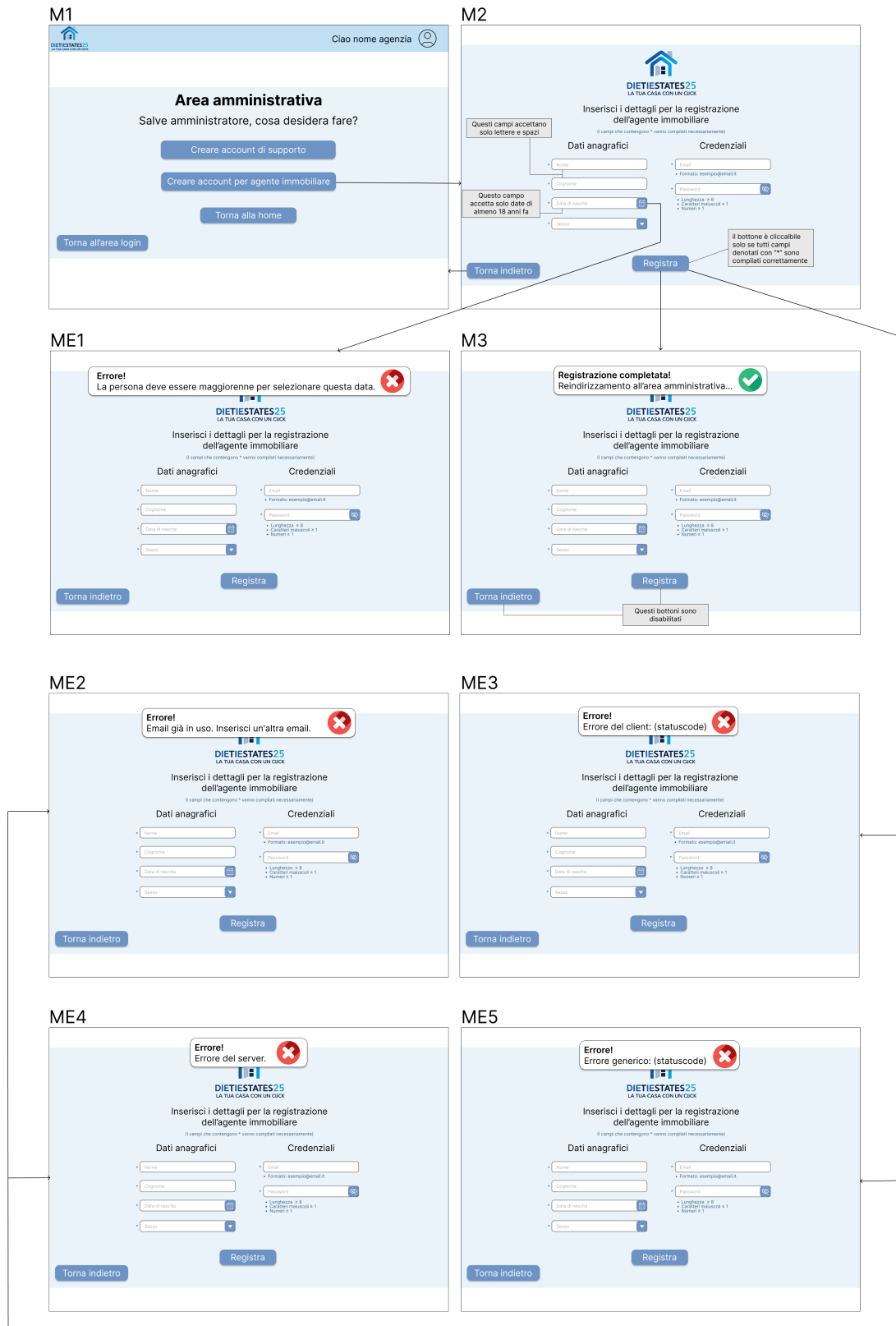
1.8.2 Visualizzare inserzioni

USE CASE #2		Creare nuova inserzione			
Goal in Context		L'agente immobiliare vuole caricare una nuova inserzione			
Preconditions		L'agente immobiliare deve aver effettuato il caso d'uso "Accedere con e-mail e password"			
Success End Condition		Il sistema tiene traccia della creazione dell'inserzione			
Failed End Condition		Il sistema non tiene traccia della creazione dell'inserzione, ma del fallimento			
Primary Actor		Agente immobiliare			
Trigger		L'agente immobiliare clicca il pulsante "Carica un nuovo immobile" in M1			
Main scenario	Step n.	Agente immobiliare	OpenLayers Map	Geoapify	Sistema
	1	Clicca "Carica un nuovo immobile" in M1			
	2				Mostra M2
	3	Clicca il bottone con il simbolo della mappa			
	4		Carica la mappa interattiva		
	5	Sceglie il punto dalla mappa interattiva e clicca "Conferma"			
	6			Verifica la presenza di luoghi di interesse nelle vicinanze dell'immobile e aggiorna le opportune checkbox	
	7	Compila i restanti dati e clicca "Avanti"			
	8				Mostra M3
	9	Controlla che tutti i dati siano corretti e clicca "Conferma"			
	8				Mostra M4 per 3 secondi e torna a M1
Extension A: L'agente immobiliare decide di voler annullare l'operazione di creazione di un inserzione	3a	Clicca "Torna indietro"			
	4a				Mostra M1 e torna a step 1 del Main scenario

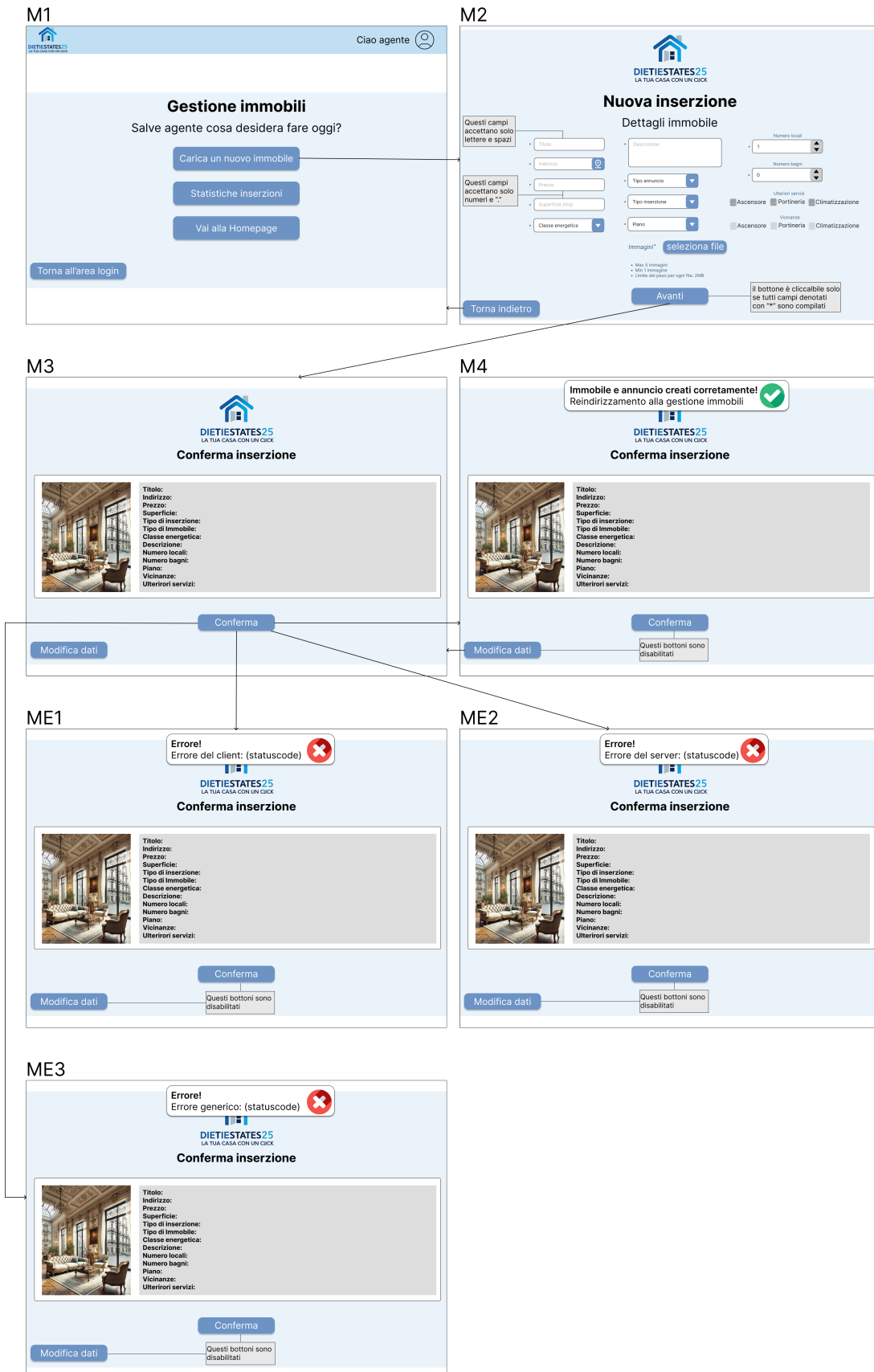
Extension B: L'agente immobiliare vuole modificare i dati appena inseriti	9b	Clicca "Modifica dati"			
	10b				Mostra M2 e torna a step 3 del Main scenario
Extension C: L'agente immobiliare decide di voler annullare l'operazione di inserimento dell'indirizzo sulla mappa	5c	Clicca "Torna indietro"			
	6c				Mostra M2 e torna a step 3 del Main scenario
Extension D: Errore da parte del client durante la conferma dell'inserzione	8d				Mostra ME1 per 3 secondi
	9d				Mostra M3 e torna a step 9 del Main scenario
Extension E: Errore da parte del server durante la conferma dell'inserzione	8e				Mostra ME2 per 3 secondi
	9e				Mostra M3 e torna a step 9 del Main scenario
Extension F: Errore generico durante la conferma dell'inserzione	8f				Mostra ME3 per 3 secondi
	9f				Mostra M3 e torna a step 9 del Main scenario

1.9 Mock-up

1.9.1 Creazione agente immobiliare



1.9.2 Creazione inserzione



1.10 Personas

In questo paragrafo ci sono esempi di persone che potrebbero essere interessate all'uso del software, da potenziali acquirenti a dipendenti di agenzie immobiliari.

1.10.1 Utenti generici



**ALFONSO
BIONDI**

- **Età:** 26 anni
- **Città:** Bologna, Italia
- **Stato sentimentale:** Fidanzato
- **Titolo di studio:** Laurea in Marketing
- **Tratti personali:** competitivo, ama il rischio, riservato

BIO

Alfonso è un ragazzo neolaureato in marketing e ha iniziato da poco a lavorare come analista in una società di Bologna. Ha deciso, insieme alla sua fidanzata, di essere pronto a intraprendere una convivenza. Sta cercando una casa in affitto a meno di 500€ al mese, vicino al luogo in cui lavora, che abbia la fermata bus vicino casa e un parco in cui poter fare attività fisica. Inoltre, essendo molto attento all'ambiente che lo circonda, vuole una casa che abbia una classe energetica di tipo A4.

GOALS

- Trovare una casa in affitto che rispetti tutte le sue richieste, senza perdere troppo tempo nel girare la città alla ricerca di una nuova casa
- Visualizzare le foto di tutte le stanze
- Impostare un prezzo massimo di affitto mensile
- Ricerare la casa in un determinato raggio

INTERESSI

- Fare esperienza lavorativa
- Fare sport all'aperto
- Dedicare tempo alla sua relazione sentimentale
- Iniziare ad essere indipendente nella vita quotidiana



**GIULIA
BIANCHI**

- **Età:** 34 anni
- **Città:** Milano, Italia
- **Stato sentimentale:** Sposata
- **Titolo di studio:** Laurea in scienze della formazione
- **Tratti personali:** Pragmatica, organizzata e molto curiosa

BIO

Giulia è un'insegnante che vive e lavora a Milano, ed ha un buon rapporto con la tecnologia. Dopo anni di affitto, ha deciso di cercare un appartamento da acquistare in una zona ben servita dai mezzi pubblici e vicina a scuole e parchi. Con il poco tempo libero a disposizione, preferisce gestire tutto da casa con il suo PC, senza dover visitare di persona diverse agenzie immobiliari. Giulia è alla ricerca di un'applicazione desktop che le permetta di visualizzare facilmente gli annunci.

GOALS

- Trovare un appartamento in vendita in una zona specifica, che sia ben collegata con il trasporto pubblico e vicina a servizi come scuole e parchi.
- Visualizzare foto e dettagli completi degli immobili per farsi un'idea chiara della casa senza doversi spostare inutilmente.
- Effettuare una ricerca avanzata e filtrare per parametri come prezzo, numero di stanze, piano, e classe energetica, in modo da risparmiare tempo nella selezione delle opzioni.

INTERESSI

- Leggere libri di Fantascienza
- Passare del tempo con la famiglia
- Stare sempre al passo con la moda

1.10.2 Agenti immobiliari



**FRANCESCO
FIORELLO**

- **Età:** 38 anni
- **Città:** Napoli, Italia
- **Stato sentimentale:** Single
- **Titolo di studio:** Diploma del liceo classico
- **Tratti personali:** ambizioso, socievole e preciso

BIO

Francesco è un agente immobiliare con oltre 12 anni di esperienza nel settore. Dopo essersi diplomato ha conseguito un corso di formazione specializzato ottenendo l'abilitazione professionale presso la Camera di Commercio. Ha già contribuito alla vendita di centinaia di immobili lavorando presso l'agenzia immobiliare "L'Immobile Dei Sogni". Grazie alla sua esperienza, riesce ad accontentare tutti i gusti dei suoi clienti, cercando di rimanere sempre nel budget specificato.

GOALS

- Incrementare la vendita delle case raggiungendo più persone possibili
- Aumentare la curiosità di un eventuale cliente verso un immobile
- Esplorare i prezzi nella zona di interesse per vendere immobili a prezzi più competitivi
- Tenere sotto controllo le statistiche degli immobili in vendita

INTERESSI

- Movimenti artistici del 1800
- Crescere lavorativamente
- Esplorare le città alla ricerca di immobili innovativi

1.10.3 Amministratori



**MARCO
ROSSI**

- **Età:** 39 anni
- **Città:** Roma, Italia
- **Stato sentimentale:** Single
- **Titolo di studio:** Laurea in gestione d'impresa
- **Tratti personali:** ambizioso, orientato ai risultati, attento ai dettagli

BIO

Marco è l'amministratore di un'agenzia immobiliare a Roma con oltre 20 anni di esperienza nel settore. È responsabile della gestione del team e si assicura che l'agenzia mantenga un alto livello di efficienza e professionalità. Marco vuole utilizzare DietiEstates25 per creare account per gli altri amministratori e per gli agenti immobiliari. Essendo molto attento alla sicurezza dei dati, apprezza particolarmente le funzionalità che garantiscono protezione e controllo sugli account, permettendo solo accessi autorizzati.

GOALS

- Creare account per utenti di supporto all'amministrazione.
- Creare account per gli agenti immobiliari.
- Rendere più visibile l'agenzia immobiliare.

INTERESSI

- Aggiornamenti e tendenze del mercato immobiliare
- Leggere il giornale quotidianamente
- Giocare a calcio
- Viaggiare

1.11 Glossario

- **API (Application Programming Interface)**
Interfaccia che consente a componenti software di comunicare tra loro attraverso un set definito di regole. Nel progetto, è usata per far interagire il front-end con il back-end.
- **Back-end**
Parte del sistema che si occupa della logica di business, della gestione dei dati e delle operazioni sul server. È indipendente dal front-end.
- **Front-end**
Interfaccia utente che consente agli utenti di interagire con il sistema. Comunica con il back-end tramite API REST.
- **REST (Representational State Transfer)**
Stile architetturale per la comunicazione tra sistemi distribuiti. Usa metodi standard del protocollo HTTP, come GET, POST, PUT e DELETE.
- **Containerizzazione**
Tecnologia che consente di eseguire applicazioni e i loro ambienti in unità isolate chiamate container. Un esempio è Docker.
- **Docker**
Strumento per la creazione e la gestione di container, che garantisce portabilità e coerenza tra ambienti di sviluppo e produzione.
- **Object-Oriented Programming (OOP)**
Paradigma di programmazione basato su oggetti, che combinano dati e metodi per operare su di essi. Linguaggi come Java, Python e C# supportano OOP.
- **UML (Unified Modeling Language)**
Linguaggio standard per la modellazione di sistemi software tramite diagrammi, come use case, diagrammi di classe e di sequenza.
- **Diagrammi di sequenza**
Diagrammi UML che descrivono il flusso di messaggi tra oggetti in un sistema nel tempo.
- **Diagrammi di classe**
Diagrammi UML che rappresentano le classi del sistema, le loro proprietà, metodi e relazioni.
- **Mock-up**
Modello grafico o prototipo statico che mostra il design di un'interfaccia utente.
- **Testing**
Processo di verifica per garantire che il software funzioni correttamente e soddisfi i requisiti. Include:
 - **Unit testing:** Verifica di singole unità o componenti del software.
 - **Usability testing:** Test per valutare la facilità d'uso del sistema.
- **Persistenza dei dati**
Meccanismo per conservare i dati su dispositivi di archiviazione (es. database) in modo che possano essere recuperati in futuro.
- **Scalabilità**
Capacità di un sistema di gestire un aumento del carico (utenti, dati, richieste) senza degradare le performance.
- **Usabilità**
Facilità con cui un utente può interagire con un sistema per raggiungere i propri obiettivi.
- **Dashboard**
Pannello di controllo interattivo che visualizza dati e statistiche in tempo reale.
- **SQL (Structured Query Language)**
Linguaggio standard per la gestione di database relazionali, utilizzato per interrogare e modificare i dati.

2 Documento di Design del Sistema

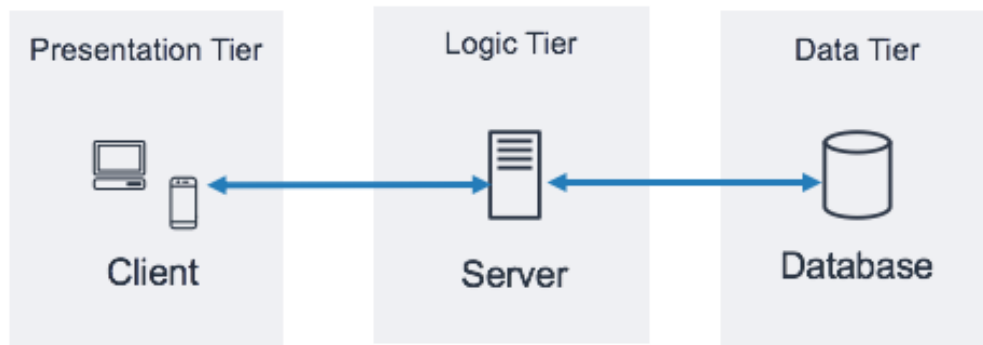
2.1 Descrizione dell'architettura proposta

L'architettura scelta per il software è quella a **tre livelli**, suddivisa in:

- **Livello di Presentazione (Front-end)**: l'interfaccia utente e le interazioni con l'utente.
- **Livello di Logica di Business (Back-end monolitico)**: le regole aziendali e l'elaborazione dei dati.
- **Livello di Persistenza (Database)**: la gestione e memorizzazione permanente dei dati.

Le principali motivazioni di questa scelta sono:

1. La possibilità di modificare una componente senza impattare sul resto del sistema.
2. Una più semplice individuazione e correzione dei bug.
3. Una comunicazione diretta tra back-end e front-end tramite API REST.
4. L'indipendenza della logica di business dall'interfaccia utente.



Architettura a tre livelli.

Fonte: https://docs.aws.amazon.com/it_it/whitepapers/latest/serverless-multi-tier-architectures-api-gateway-lambda/images/image2.png

2.1.1 Perché un back-end monolitico?

Un **back-end monolitico** può gestire efficacemente carichi moderati senza problemi di scalabilità orizzontale. La nostra scelta è ricaduta su quest'architettura perché il software *DietiEstates25* non richiede un elevato carico di richieste né una logica complessa (in caso contrario, sarebbe stata più indicata un'architettura a microservizi, pur mantenendo la possibilità di migrare in futuro). Con questa architettura:

- **Tutta la logica di business** risiede in un'unica applicazione, riducendo la complessità generale del sistema.
- Non è necessario gestire la comunicazione tra microservizi, evitando overhead nella comunicazione e semplificando lo sviluppo.
- **Dati e logica di business** sono strettamente legati, il che consente una riduzione della latenza nelle risposte alle richieste degli utenti.

2.1.2 Livello di Presentazione (Front-end)

Questo livello rappresenta l'interfaccia utente con cui gli utenti interagiscono direttamente. La sua funzione principale è mostrare i dati provenienti dal back-end e raccogliere input dagli utenti, inviando richieste tramite API REST.

Funzioni principali:

- Visualizzare dati come elenco di immobili, mappe, e statistiche.
- Gestire interazioni come login e ricerche avanzate.
- Presentare messaggi di errore all'utente

Pattern utilizzato: Il pattern scelto per la realizzazione di questo livello è **MVC (Model-View-Controller)**.

- **Model:** rappresenta i dati e lo stato dell'applicazione. È responsabile della gestione dei dati.
- **View:** gestisce la presentazione e l'interazione con l'utente. Mostra i dati e lo stato dell'applicazione. Nel nostro caso verrà sviluppato tramite **JavaFX**.
- **Controller:** si occupa di aggiornare la View e inoltrare le azioni al Model, tramite la comunicazione con il Service.
- **Service:** si occupa di comunicare mediante richieste HTTP tramite API REST con il back-end.

Vantaggi:

- **Separazione di responsabilità:** ogni livello ha un ruolo ben definito, migliorando leggibilità e manutenibilità.
- **Manutenibilità:** ogni componente è modificabile senza impattare sugli altri livelli.
- **Testabilità:** ogni componente può essere testato in modo indipendente rendendo i test unitari più semplici.
- **Gestione delle richieste REST:** il Service gestisce le richieste REST passando i dati al Controller, che le inoltra al Model, aggiornando la View.

2.1.3 Livello di Logica di Business (Back-end)

Contiene la logica applicativa, gestisce le regole aziendali, elabora i dati e si interfaccia con il database. Espone API REST per ricevere richieste dal front-end e interagisce con il database per accedere o modificare i dati. La tecnologia scelta è Spring.

Funzioni principali:

- **Autenticazione degli utenti:** login/registrazione con credenziali o provider terzi (Google e GitHub).
- **Gestione delle inserzioni immobiliari:** creazione e ricerca avanzata.
- **Statistiche e reportistica:** calcolo e generazione di report esportabili in PDF e CSV.
- **Integrazione con servizi esterni:** comunicazione con API come Geoapify per la geolocalizzazione e con OpenLayers Map per il caricamento delle mappe interattive.

Pattern utilizzato: Il pattern scelto per la realizzazione di questo livello è **API-Business-Data**.

- **Api**
 - **Controller:** espone le API del sistema, riceve le richieste HTTP e le passa al livello Service ritornando le risposte HTTP al client in modo standardizzato.
 - **DTO:** classi che servono a trasferire dati da e verso il front-end.
 - **Exception:** classi che gestiscono e centralizzano le eccezioni.
- **Business**
 - **Service:** implementa la logica di business. Riceve i dati dal Controller, applica la logica richiesta e interagisce con il Repository per accedere ai dati e salvarli.
 - **Entity:** classi che rappresentano il modello persistente e che mappano direttamente le tabelle del database.
- **Data**
 - **Config:** classi di configurazione specifiche di Spring, per il livello di persistenza.
 - **Repository:** gestisce tutte le operazioni sul database ed è l'unico ad interagirci direttamente.

Vantaggi:

- **Separazione di responsabilità:** ogni livello ha un ruolo ben definito, migliorando leggibilità e manutenibilità.
- **Manutenibilità:** ogni componente è modificabile senza impattare sugli altri livelli.
- **Testabilità:** ogni componente può essere testato in modo indipendente rendendo i test unitari più semplici.
- **Scalabilità interna:** è possibile aggiungere nuove funzionalità senza dover modificare gran parte del codice.

Migrazione del database

Per avere uno storico del database abbiamo usato **FlyWay** che consente di poter effettuare la migrazione del database ed avere un elenco di versioni con tutte le modifiche apportate al database.

2.1.4 Livello di Persistenza (Database)

Questo livello si occupa della memorizzazione permanente dei dati (comunicandoli al back-end su richiesta) e della gestione efficiente e sicura delle informazioni relative al sistema.

Tecnologia utilizzata: la scelta per l'implementazione del database è stata fatta con l'uso di **PostgreSQL** che riceve richieste tramite il Repository del back-end.

Vantaggi:

- **Centralizzazione dell'accesso ai dati:** tutte le operazioni sui dati passano attraverso il livello di Persistenza, consentendo un controllo centralizzato e uniforme sulle query.

2.2 Tecnologie usate

2.2.1 JavaFX

JavaFX consente di definire l'interfaccia grafica tramite file FXML, separando la struttura della UI dal codice Java; inoltre c'è a disposizione il tool SceneBuilder che velocizza il processo iniziale per la creazione delle schermate. **Motivi della scelta:**

- **Supporto a CSS:** offre CSS per lo styling e supporta animazioni avanzate, consentendo la creazione di interfacce utente moderne e dinamiche. Inoltre si rimuove molto codice duplicato per gli stili condivisi nelle varie schermate
- **Portabilità:** è basato su Java, quindi può essere eseguito su Windows, macOS e Linux senza modifiche.
- **Separazione delle responsabilità:** supporta il pattern MVC, separando la logica di presentazione dalla logica di business.



Fonte: Wikipedia

2.2.2 Spring

Spring è un framework Java, quindi si integra perfettamente con il front-end in JavaFX. È possibile riusare parti del codice condivise tra front-end e back-end senza problemi. **Motivi della scelta:**

- **Supporto API REST:** offre Spring Web, che facilita la creazione di API REST con annotazioni come `@RestController` e `@RequestMapping`.
- **Spring Data JPA:** Spring Data JPA semplifica l'interazione con PostgreSQL, riducendo il boilerplate necessario per le operazioni CRUD.
- **Dependency Injection:** Spring utilizza Dependency Injection (DI) per gestire le dipendenze in modo automatico.
- **Sicurezza:** Spring Security offre un framework robusto per l'autenticazione e l'autorizzazione.
- **Transazioni:** supporta la gestione delle transazioni grazie all'annotazione `@Transactional`, garantendo il rispetto delle proprietà ACID nel database PostgreSQL.

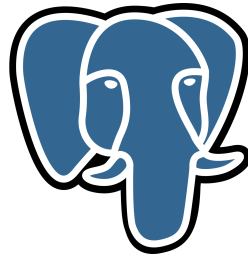


Fonte: Wikipedia

2.2.3 PostgreSQL

La scelta dell'uso di PostgreSQL come database è avvenuta per **i seguenti motivi**:

- **Proprietà ACID**: PostgreSQL è completamente conforme agli standard SQL e garantisce proprietà ACID (Atomicità, Consistenza, Isolamento, Durabilità) assicurando transazioni sicure e coerenti, riducendo il rischio di perdita o corruzione dei dati.
- **Scalabilità**: la scelta dell'uso di un database relazionale rende semplice la scalabilità verticale, oppure, in caso di molte letture e poche scritture, la scelta migliore sarebbe creare più istanze sfruttando il pattern *"Master-Slave"*.



Fonte: Wikipedia

2.2.4 Amazon Web Services

AWS è stato scelto come provider cloud per l'hosting del back-end e del database, sfruttando rispettivamente i servizi **Amazon EC2** e **Amazon RDS**. Questa scelta è motivata **dai seguenti vantaggi**:

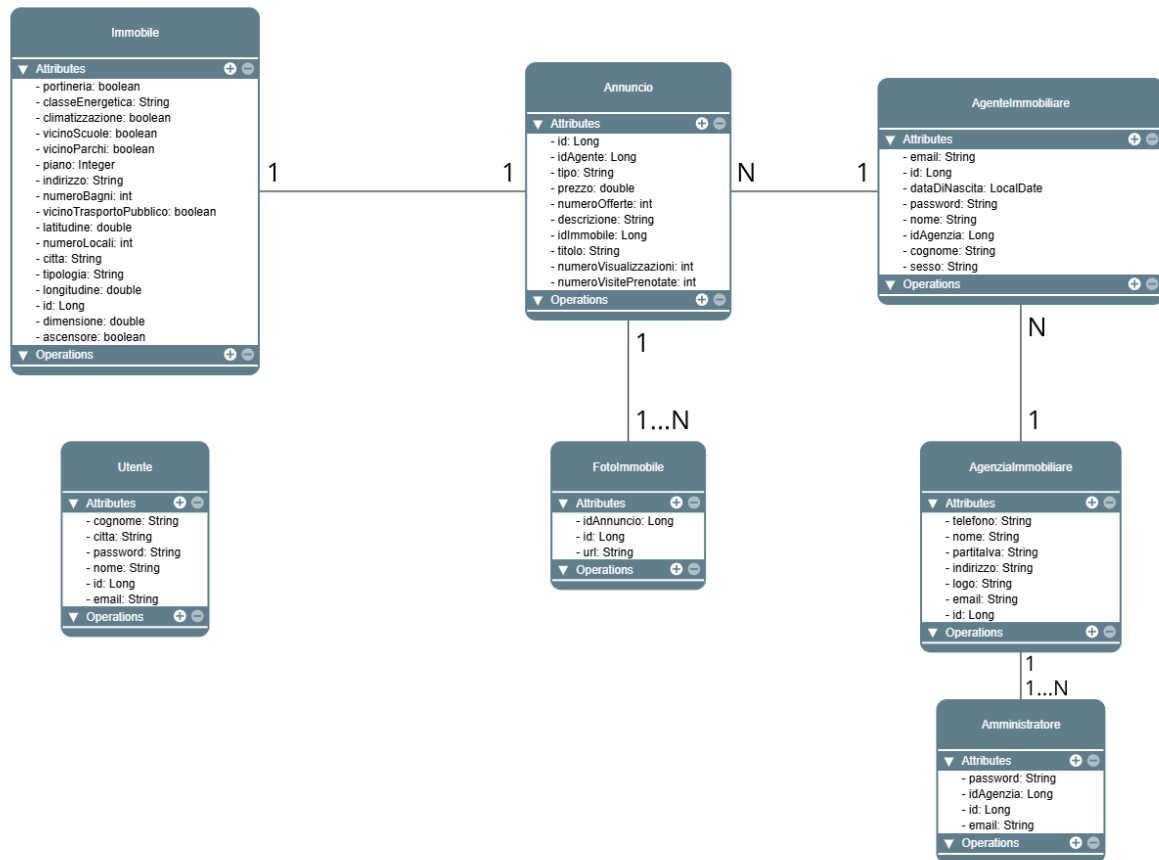
- **Scalabilità**: AWS permette di scalare facilmente le risorse in base al carico di lavoro, garantendo flessibilità e adattabilità alla crescita dell'applicazione.
- **Affidabilità**: EC2 e RDS sono progettati per garantire alta disponibilità e tolleranza ai guasti, con backup automatici e meccanismi di ripristino.
- **Sicurezza**: AWS fornisce strumenti avanzati di sicurezza come VPC, gruppi di sicurezza, crittografia dei dati e gestione degli accessi tramite IAM.
- **Gestione semplificata del database**: Con Amazon RDS, le attività di manutenzione come aggiornamenti, backup e ripristini vengono automatizzate, riducendo l'onere amministrativo.
- **Integrazione con altri servizi AWS**: L'infrastruttura può essere facilmente estesa con altri servizi come S3 per lo storage o CloudWatch per il monitoraggio.



Fonte: Wikipedia

2.3 Schema di persistenza dei dati

Il **diagramma UML** di seguito mostra le **entità** principali e le loro **relazioni**. È progettato per fornire una visione chiara della struttura dei dati e delle interazioni tra le diverse entità del sistema.



2.3.1 Entità

- **Immobile**: Rappresenta un immobile con vari attributi come indirizzo, dimensioni, tipologia, posizione geografica (latitudine, longitudine) e dettagli relativi alle caratteristiche (numero di locali, bagni, presenza di ascensore, classe energetica, ecc.).
- **Annuncio**: Rappresenta un annuncio di vendita o affitto di un immobile. Contiene informazioni come il tipo di annuncio, il prezzo, la descrizione, il numero di visualizzazioni, il numero di offerte, e un riferimento all'immobile a cui si riferisce inoltre è associato a un agente immobiliare.
- **AgenteImmobiliare**: Rappresenta un agente immobiliare, con attributi quali nome, cognome, email, data di nascita, password, sesso e un riferimento all'agenzia immobiliare a cui appartiene.
- **AgenziaImmobiliare**: Rappresenta un'agenzia immobiliare, con attributi quali telefono, nome, partita IVA, indirizzo, l'URL per accedere al logo e email.
- **Utente**: Rappresenta un utente generico del sistema, con attributi come nome, cognome, città, email e password. Questa entità viene utilizzata per rappresentare utenti registrati che interagiscono con il sistema.
- **Amministratore**: Rappresenta un amministratore di un'agenzia immobiliare, con attributi per la gestione dell'autenticazione (password, email) e un riferimento all'agenzia immobiliare che gestisce.
- **FotoImmobiliare**: Rappresenta un'immagine associata a un annuncio immobiliare, contenente un riferimento all'annuncio e l'URL per accedere all'immagine.

2.3.2 Associazioni

- Un **Immobile** ha uno e un solo **Annuncio**. (Relazione 1-1)
- Un **Annuncio** può avere una o più **FotoImmobile**. (Relazione 1-N)
- Un **Annuncio** è associato a un **AgenteImmobiliare**. Molti annunci possono essere associati a uno stesso agente (Relazione N-1).
- Un **AgenteImmobiliare** appartiene a una **AgenziaImmobiliare**. Molti agenti appartengono a una stessa agenzia (Relazione N-1).
- Un **AgenziaImmobiliare** può avere molti **Amministratore**. (Relazione 1-N)

2.4 Scelte di design dell'interfaccia utente

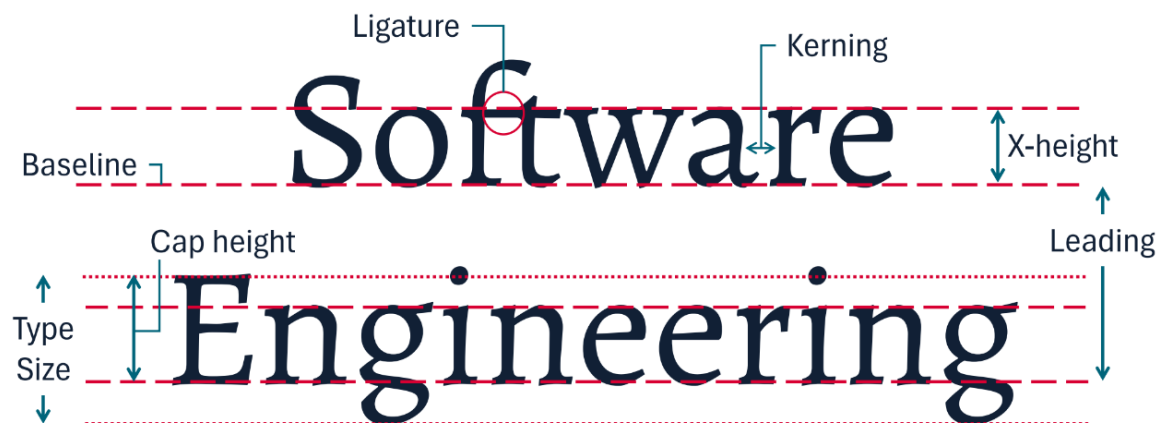
In questa sezione parleremo delle scelte di design dell'interfaccia utente adottate, motivando tutte le scelte prese. Ci siamo impegnati per rendere l'interfaccia utente il più possibile :

- **Coerente**, non ci sono discrepanze tra azioni simili
- **Usabile** da una vasta gamma di utenti
- **Intuitiva** ovvero offrendo riscontri informativi
- **Priva di errori** non voluti da parte dell'utente
- **Reversibile** ovvero potendo annullare sempre le operazioni in corso

2.4.1 Scelta del font

Il font Verdana è stato scelto come font principale per la sua eccellente leggibilità sullo schermo, la sua chiarezza e neutralità, e la sua disponibilità diffusa. Il design sans-serif lo rende facile da leggere in diverse dimensioni e su diversi dispositivi.

- **Leggibilità Eccellente sullo Schermo:** Verdana è progettato specificamente per la visualizzazione su schermi, con un'altezza-x generosa che facilita la lettura anche in dimensioni ridotte e su diversi dispositivi.
- **Chiarezza e Neutralità:** Come font sans-serif, Verdana offre una forma delle lettere pulita e semplice, garantendo chiarezza e un aspetto professionale. La sua neutralità evita distrazioni e contribuisce a un'esperienza utente affidabile.
- **Disponibilità Diffusa:** Verdana è un font standard incluso in molti sistemi operativi, riducendo il rischio di problemi di visualizzazione dovuti a sostituzioni di font e migliorando i tempi di caricamento della pagina evitando la necessità di font personalizzati.

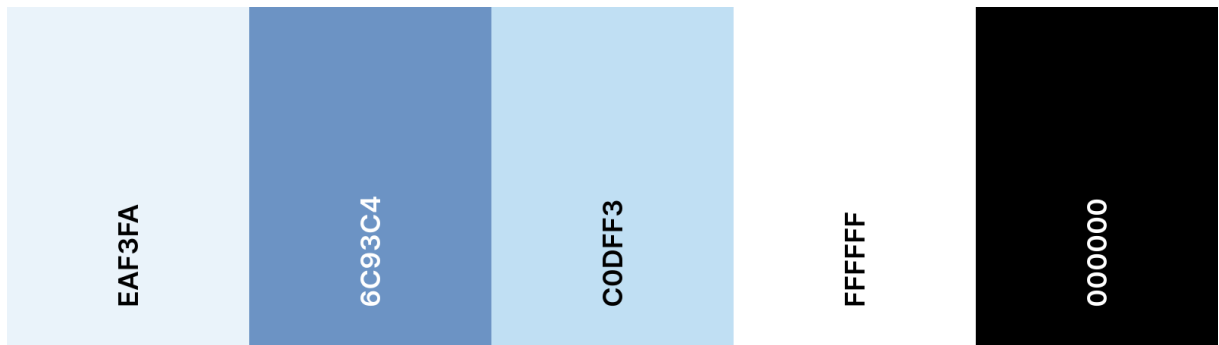


Fonte: Luigi Libero Lucio Starace, Ph.D. - University of Naples Federico II - Software Engineering Course - Lecture 15 - Colors, Typography and Gestalt

2.4.2 Scelta dei Colori

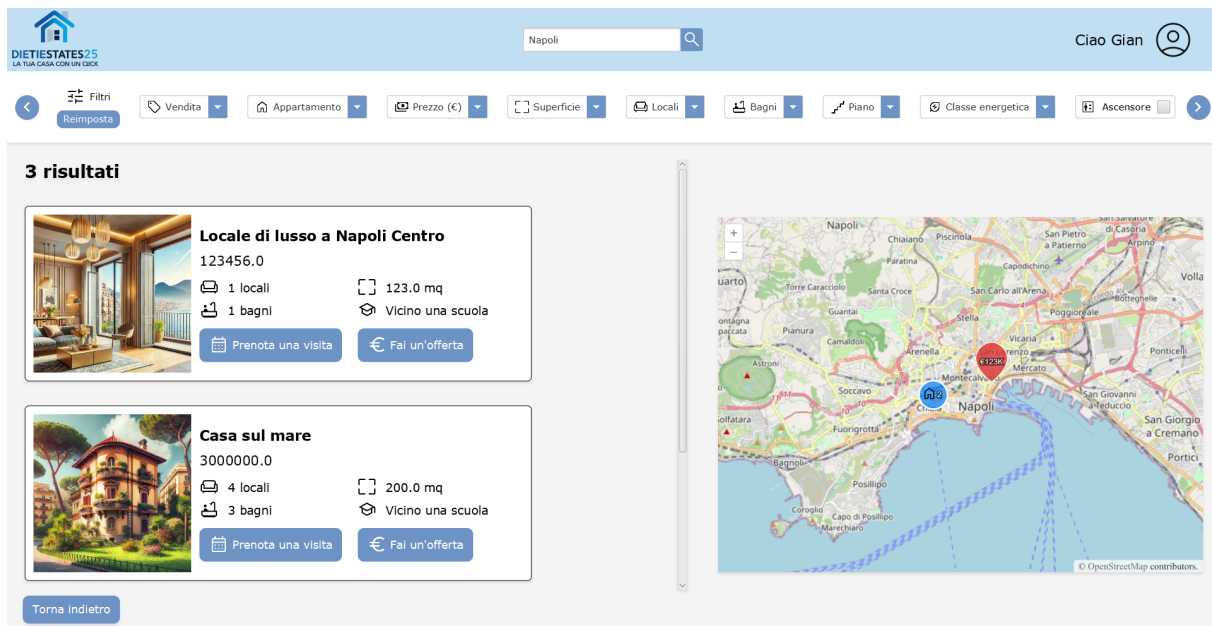
La palette di colori è stata attentamente selezionata per creare un'esperienza utente accogliente, professionale e intuitiva. L'obiettivo è quello di trasmettere fiducia, chiarezza e accessibilità.

- **Sfondo Primario: #EAF3FA (Azzurro Chiaro):** Questo colore azzurro chiaro è stato scelto per la sua capacità di creare un'atmosfera serena. Richiama la tranquillità e l'affidabilità, offrendo una base neutra che non distrae l'utente dai contenuti principali. Il suo basso contrasto con il bianco (sfondo secondario) favorisce la leggibilità del testo.
- **Sfondo Secondario: Bianco:** Il bianco è utilizzato per sezioni specifiche di contenuto e moduli. Fornisce pulizia visiva e aiuta a separare gli elementi dell'interfaccia, migliorando la gerarchia delle informazioni.
- **Bottoni: #6C93C4 (Blu tenue):** Il blu tenue per i bottoni è stato scelto per la sua associazione con la fiducia e la professionalità. È un colore non invadente ma ben visibile che attira l'attenzione dell'utente verso le azioni principali da compiere. La tonalità di blu si integra armoniosamente con lo sfondo azzurro, creando un'interfaccia coesa.
- **Scritte nei Bottoni: Bianco:** L'utilizzo del bianco per il testo nei bottoni garantisce un elevato contrasto con lo sfondo blu, rendendo i bottoni facilmente leggibili e riconoscibili.
- **Scritte in Generale: Nero:** Il nero per le scritte in generale assicura la massima leggibilità e chiarezza su entrambi gli sfondi (azzurro e bianco).
- **Navbar: #C0DFF3 (Azzurro leggermente più scuro del primario):** Una navbar leggermente più scura dello sfondo primario fornisce una distinzione visiva senza interrompere la continuità estetica.



Palette dei colori

2.4.3 Esempio di principi di Gestalt usati



Schermata di ricerca immobili

- **Proximity:**

- Raggruppamento delle informazioni degli immobili: Gli elementi relativi a ciascun immobile (immagine, titolo, prezzo, dettagli come numero di locali/bagni, superficie, servizi vicini, bottoni di azione) sono raggruppati visivamente vicini tra loro. Questo suggerisce immediatamente che tutti questi elementi appartengono alla stessa "unità".
- Filtri di ricerca: I vari filtri (Vendita, Appartamento, Prezzo, etc.) sono posti uno accanto all'altro, comunicando che fanno parte di una stessa funzione: il filtraggio.

- **Similarity:**

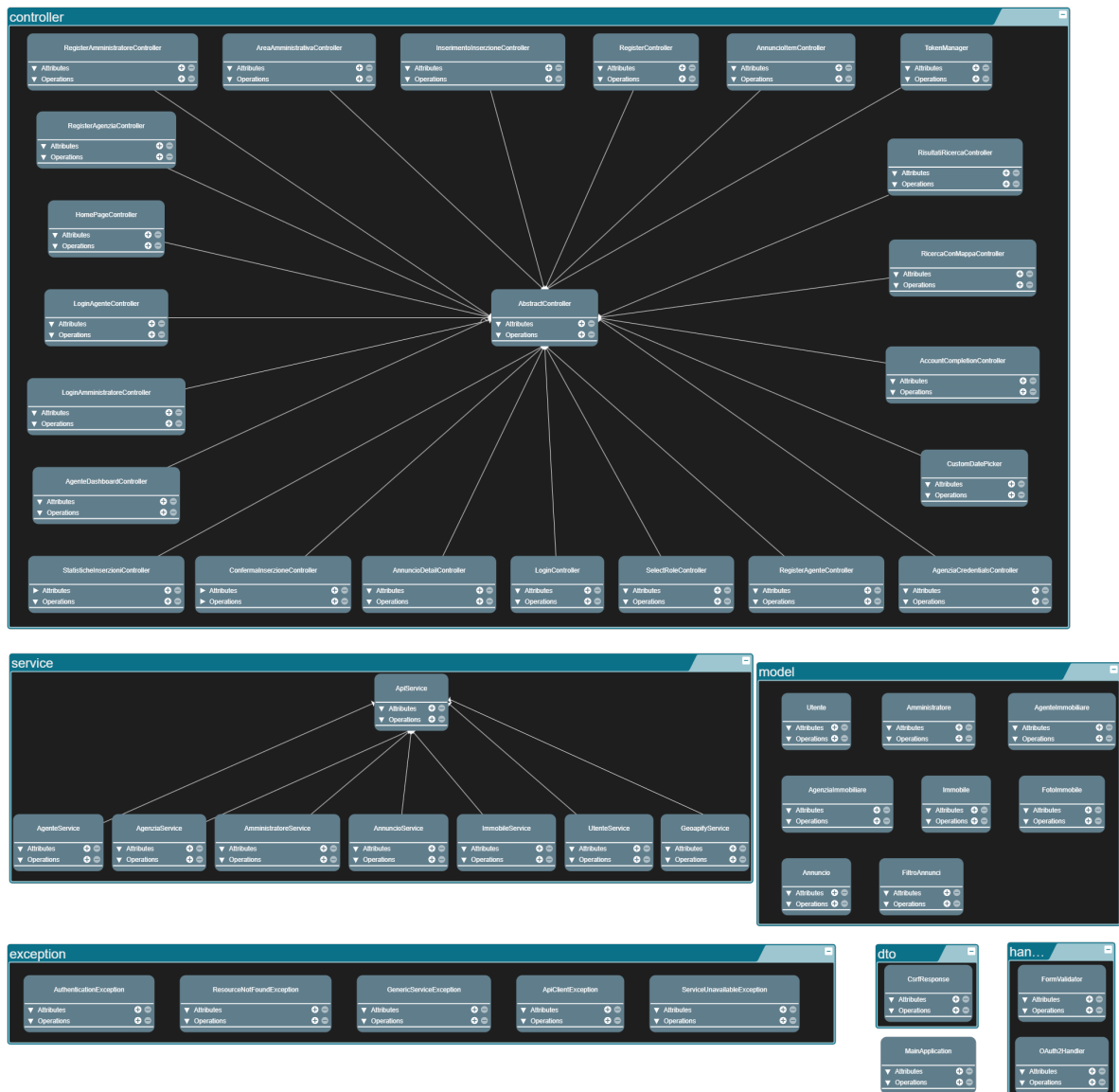
- Stile dei bottoni: I bottoni "Prenota una visita" e "Fai un'offerta" hanno lo stesso colore, forma e stile. Questo suggerisce di pari importanza che l'utente può intraprendere.
- Icone dei filtri di ricerca: Le icone utilizzate nei filtri di ricerca sono dello stesso stile. Questo indica che svolgono una funzione simile, ovvero filtrare i risultati.
- Nella mappa quando due marker sono troppo vicini vengono raggruppati in uno solo con lo stile diverso.

- **Common Region:**

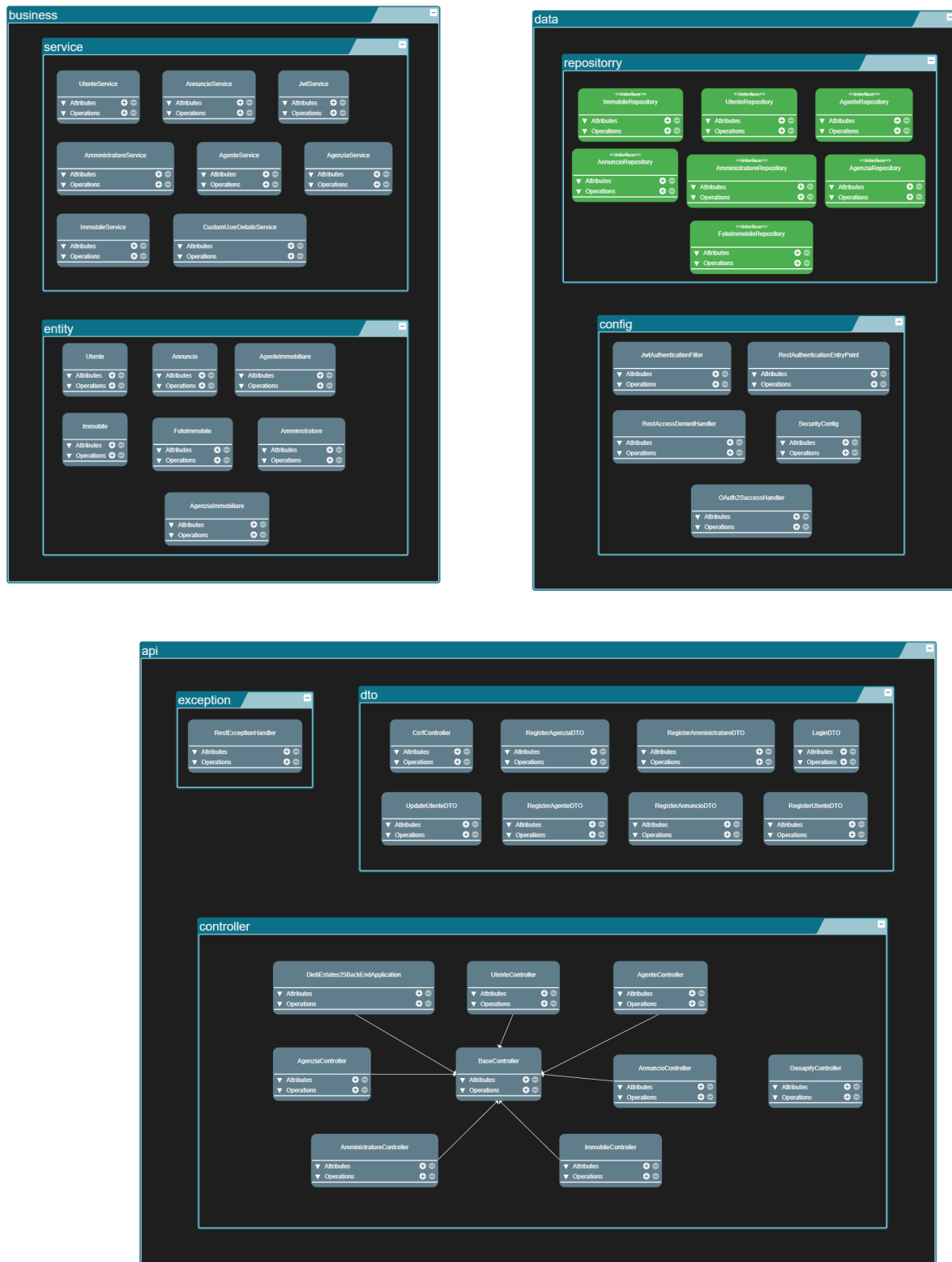
- I box che contengono le informazioni di ciascun immobile sono figure complete e delimitate da bordi, raggruppando tutte le informazioni correlate.

2.5 Diagramma delle classi di design

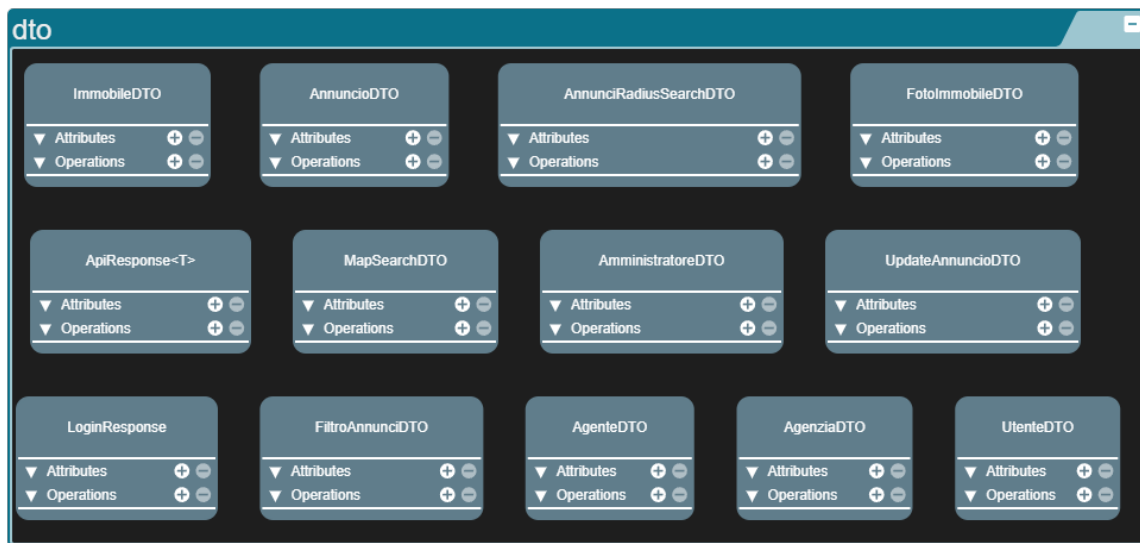
2.5.1 Front-end



2.5.2 Back-end



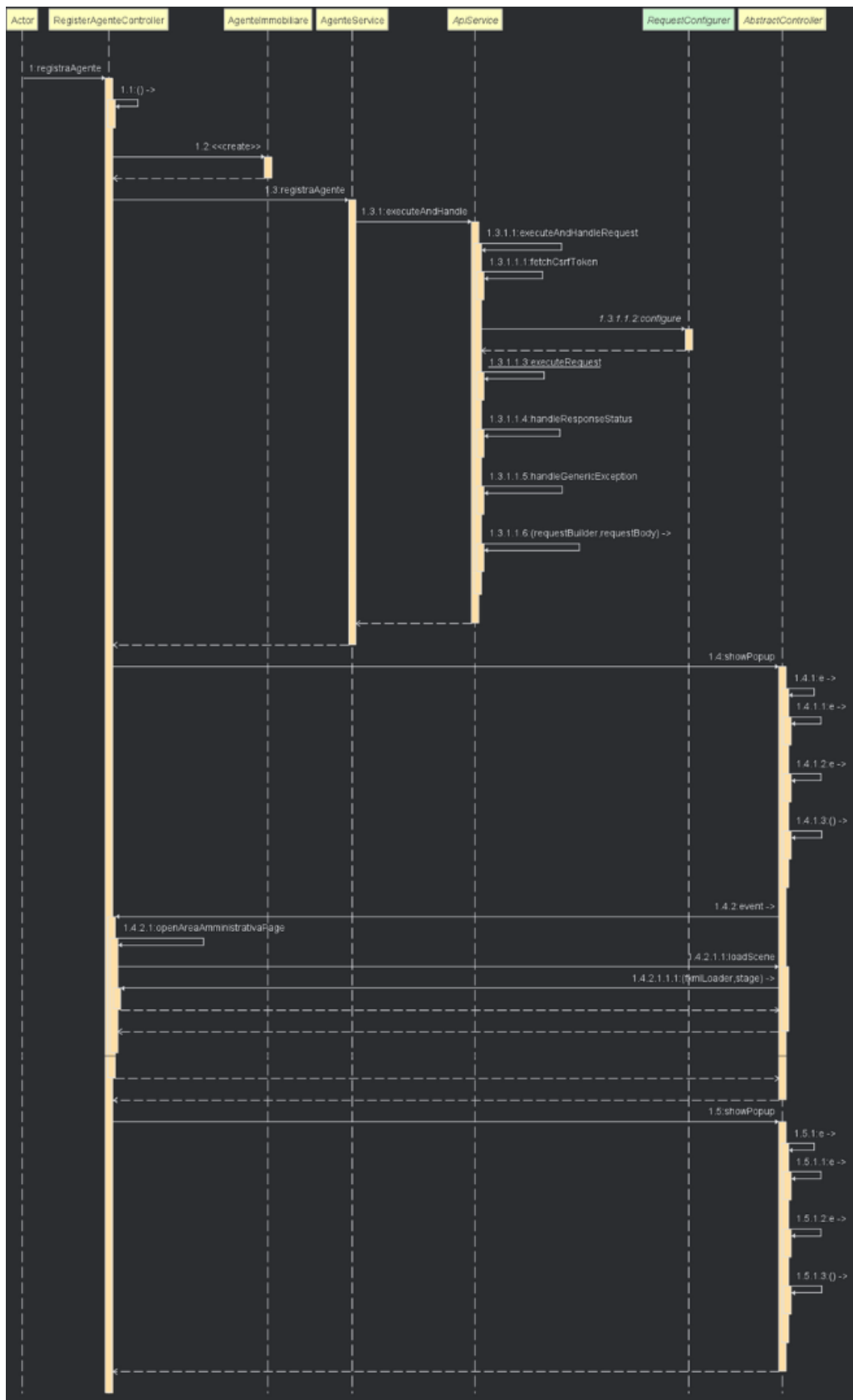
2.5.3 Shared-Library



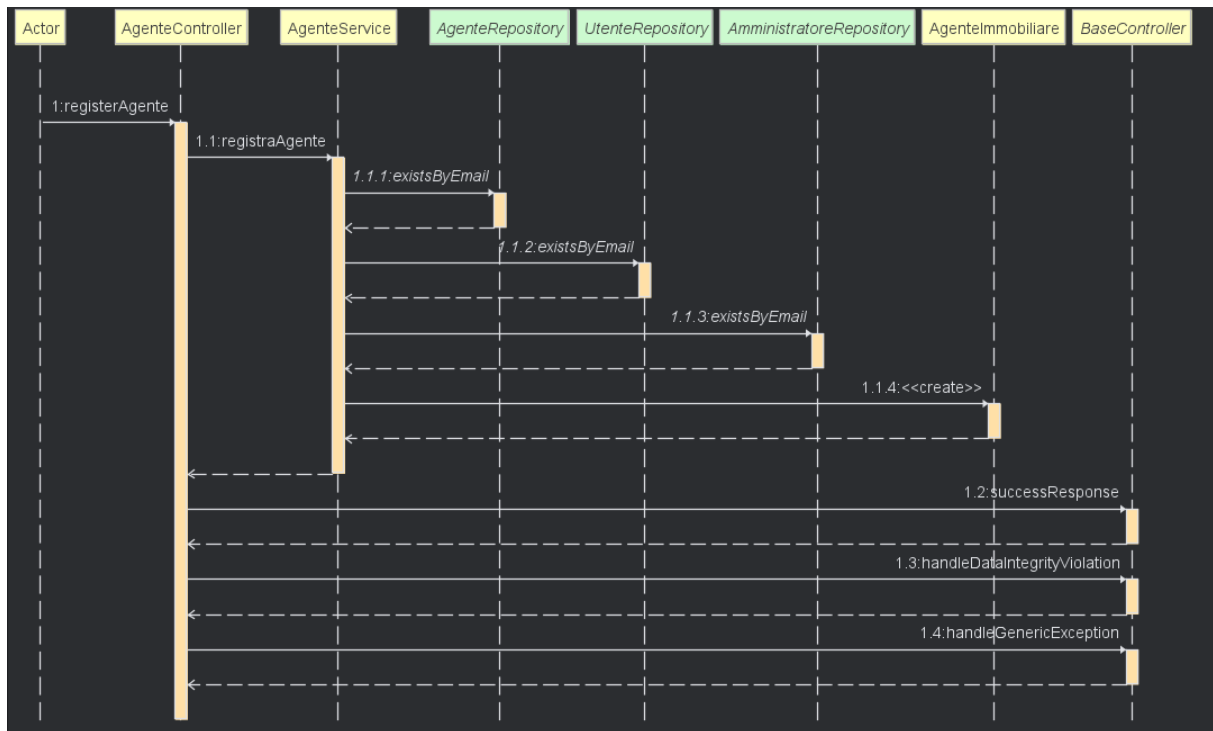
2.6 Diagrammi di sequenza di design

2.6.1 Caso d'uso: registrazione di un agente

Lato Front-end:



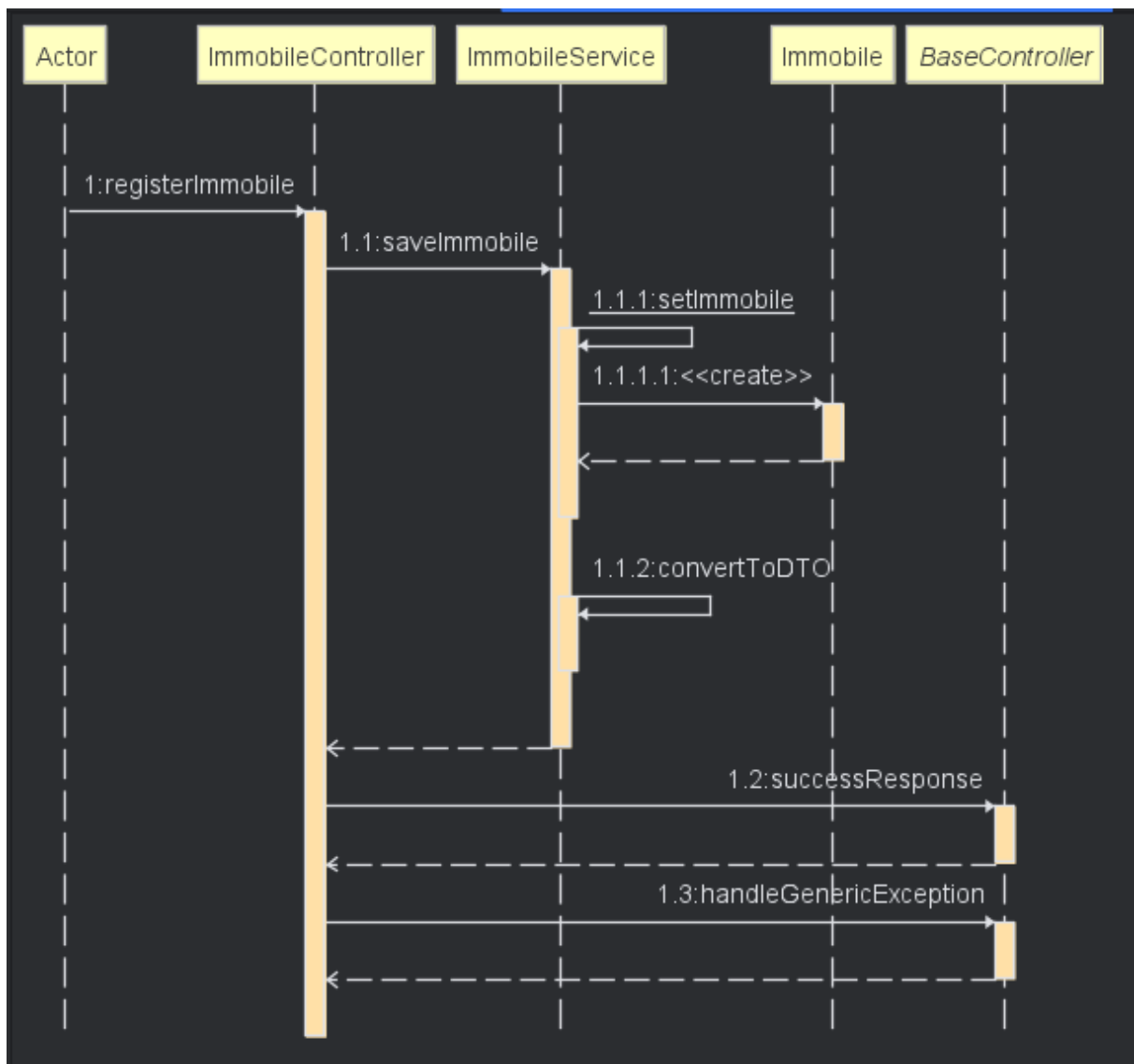
Lato Back-end:



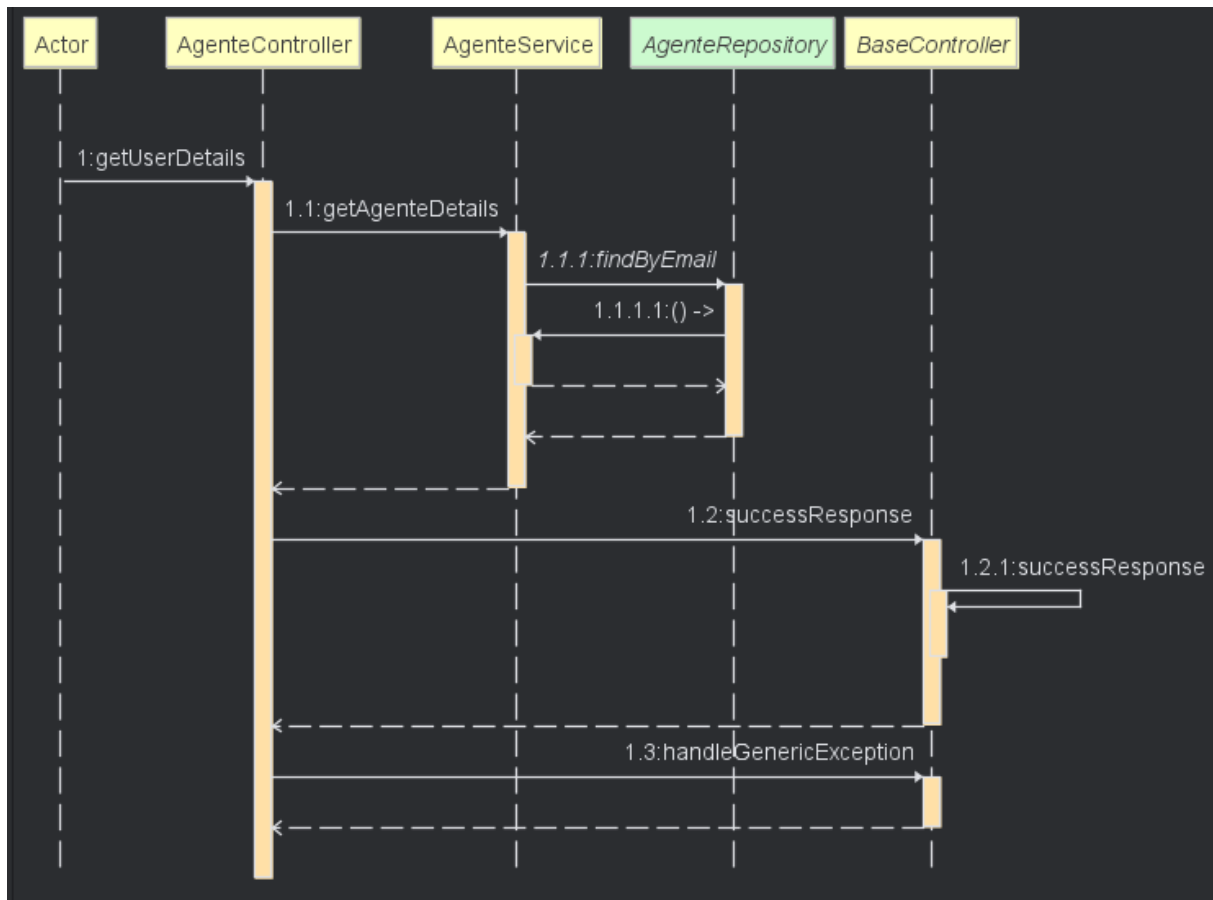
Lato Front-end:



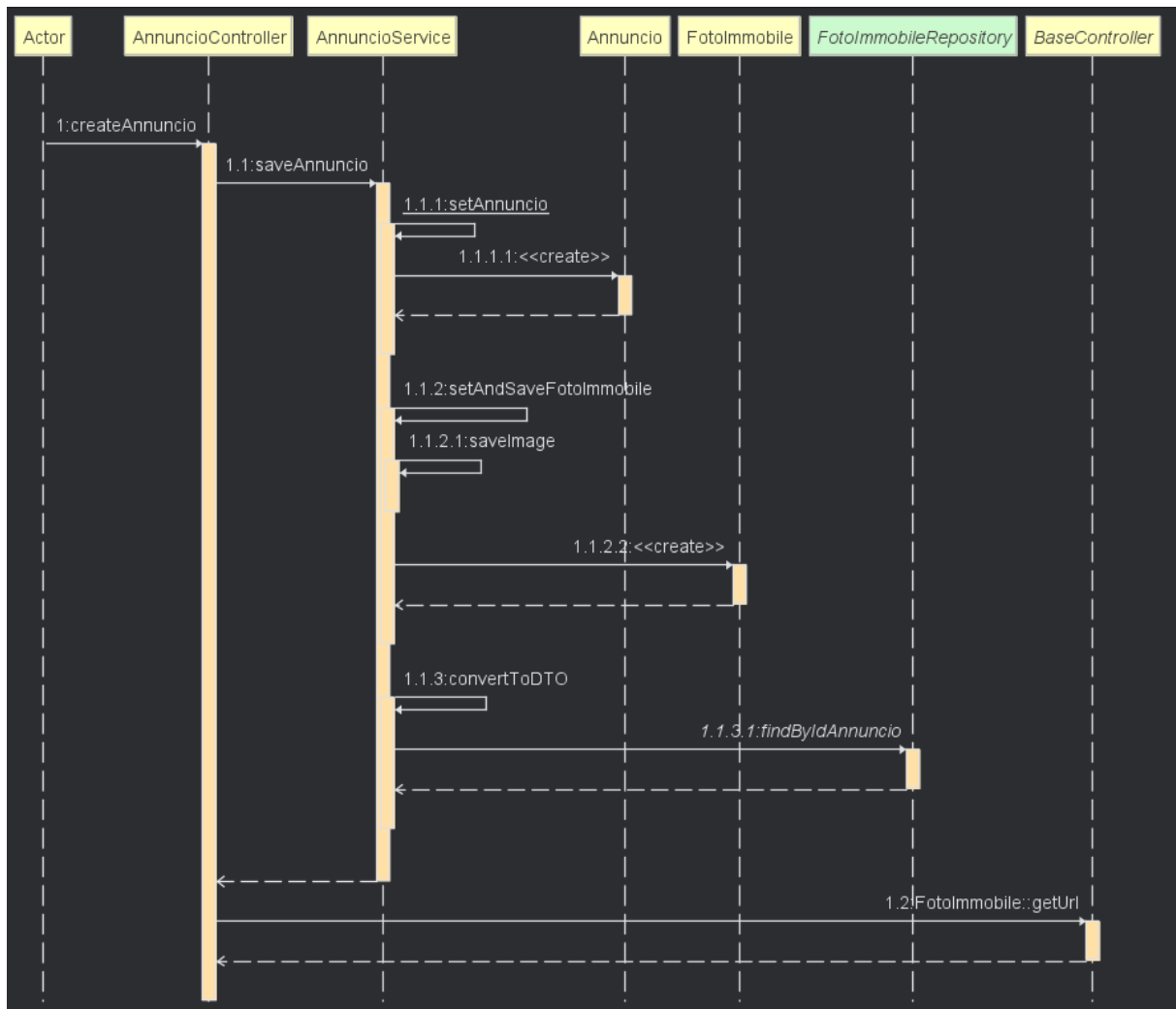
Lato Back-end (creazione immobile):



Lato Back-end (recupero dati agente):



Lato Back-end (creazione annuncio):



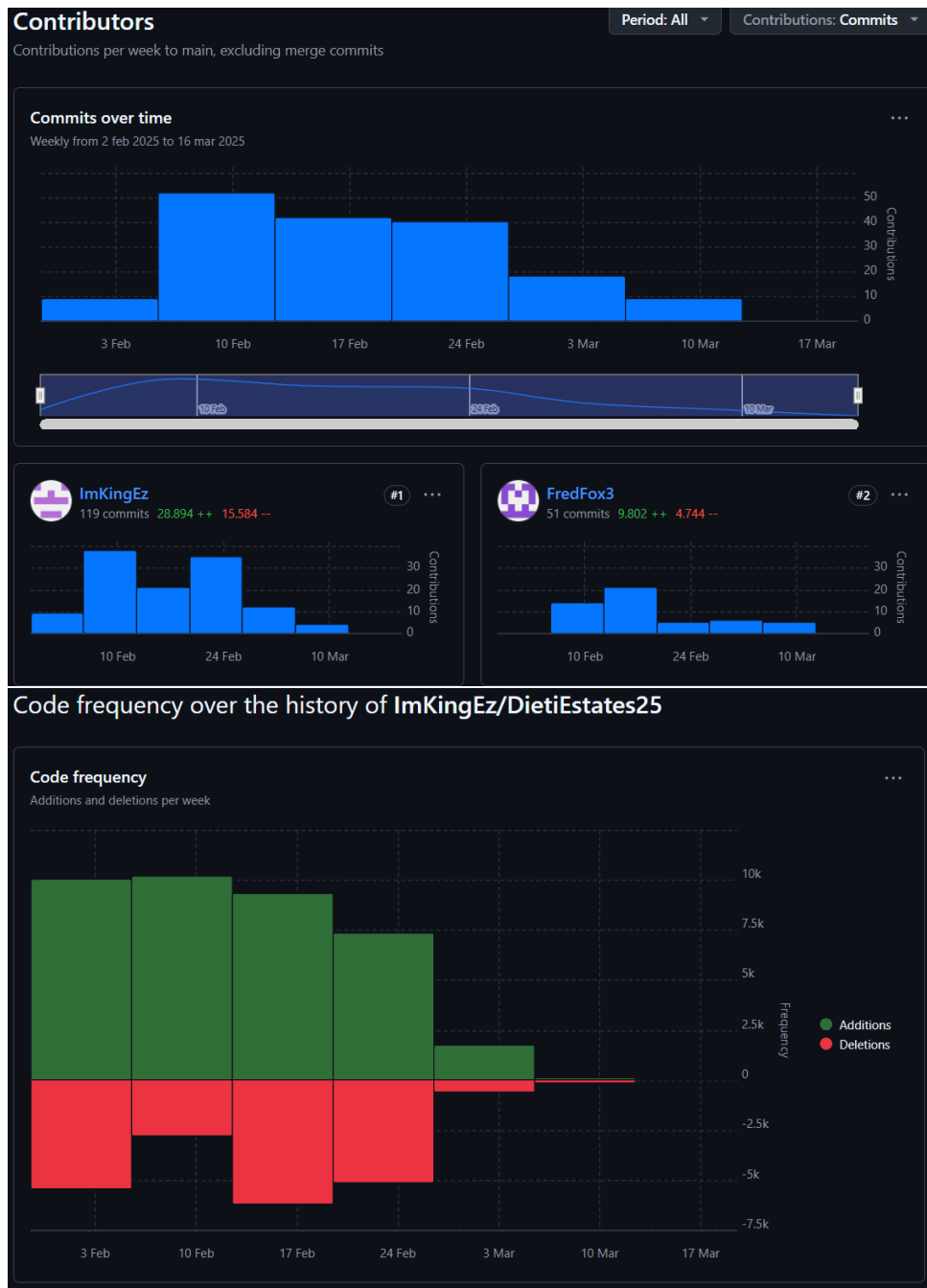
3 Processo di sviluppo

3.1 Codice sorgente, pom, Dockerfile e docker-compose

Tutto il codice sorgente del progetto è disponibile al seguente repository di github:
<https://github.com/ImKingEz/DietiEstates25>

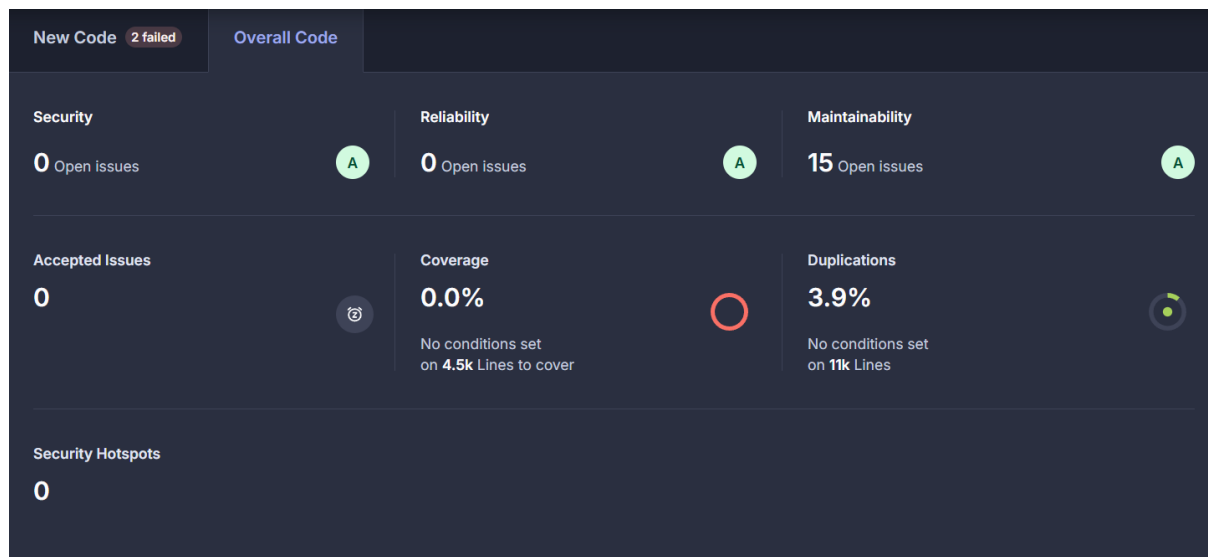
3.2 Statistiche di GitHub

Durante lo sviluppo, come strumento di **versioning**, abbiamo scelto di usare **GitHub**. Inizialmente abbiamo lavorato su un'unica macchina per creare lo scheletro principale del programma e, una volta reso funzionante, abbiamo lavorato su diversi **branch** in modo da poter dividere il lavoro sviluppando parti del software differenti. Per ogni **pull request** organizzavamo una discussione del codice prodotto in modo da mostrare all'altra persona il risultato e confrontandosi in modo da trovare possibili miglioramenti. Di seguito sono mostrate le statistiche dei contributi al repository.



3.3 Report di qualità del codice

Per monitorare la qualità del codice durante lo sviluppo abbiamo usato **SonarQube** tramite Docker per un controllo locale e abbiamo impostato un controllo automatico tramite **SonarQube Cloud** per ogni modifica fatta al branch main del repository. Di seguito sono riportati i dati dell'ultima scansione effettuata su tutto il codice.



4 Testing e valutazione dell'usabilità

4.1 Piano di testing

Per la creazione dei test dei punti descritti nel paragrafo 1.9 è stata adottata una strategia di test *Black-Box* basata sull'approccio **R-WECT**.

La strategia *Black-Box* è stata scelta per concentrarsi sui requisiti funzionali del sistema, testando i servizi dal punto di vista dell'utente, senza necessità di conoscere i dettagli implementativi interni. L'approccio R-WECT, nello specifico, permette di:

- **Definire Classi di Equivalenza:** I dati di input per i metodi sono stati suddivisi in classi di equivalenza valide e non valide. Testare un rappresentante di ogni classe permette di coprire un'ampia gamma di input possibili, riducendo il numero totale di test necessari.
- **Identificare Valori Limite:** Per ogni classe di equivalenza, sono stati identificati i valori limite. Testare questi valori limite è cruciale per individuare errori di gestione dei limiti.

Le tabelle seguenti descrivono le classi di equivalenza identificate per ciascun attributo utilizzato nei test:

4.1.1 Metodo `updateAnnuncioStats`

Tabella 2: Classi di Equivalenza per `updateAnnuncioStats`

Attributo	Classe di Equivalenza	Valori di Test
idImmobile	Valido (CE1: > 0)	1, 5, 100
	Non valido (CE2: $= 0$)	0
	Non valido (CE3: < 0)	-1
	Non valido (CE4: <code>null</code>)	<code>null</code>
tipoAggiornamento	Valido (CE1: "Visualizzazione")	"visualizzazione"
	Valido (CE2: "Visita")	"visita"
	Valido (CE3: "Offerta")	"offerta"
	Non valido (CE4: <code>null</code>)	<code>null</code>
	Non valido (CE5: Stringa vuota)	""
	Non valido (CE6: Valore non ammesso)	"random"

Codice test

```
1 @ExtendWith(MockitoExtension.class)
2 class AnnuncioServiceTest {
3
4     @InjectMocks
5     @Spy
6     private AnnuncioService annuncioService;
7
8
9     @Nested
10    @DisplayName("Casi di aggiornamento non validi")
11    class UpdateStatsFailureTests {
12
13        static Stream<Arguments> failureCases() {
14            return Stream.of(
15                Arguments.of(ID_IMMOBILE_ZERO,
16                    TIPO_AGGIORNAMENTO_VISUALIZZAZIONE),
17                Arguments.of(ID_IMMOBILE_NEGATIVE,
18                    TIPO_AGGIORNAMENTO_VISITA),
19                Arguments.of(ID_IMMOBILE_NULL, TIPO_AGGIORNAMENTO_OFFERTA),
20                Arguments.of(ID_IMMOBILE_NOMINAL, TIPO_AGGIORNAMENTO_NULL),
21                Arguments.of(ID_IMMOBILE_MINIMAL, TIPO_AGGIORNAMENTO_EMPTY),
22                Arguments.of(ID_IMMOBILE_NOMINAL, TIPO_AGGIORNAMENTO_RANDOM)
23            );
24        }
25
26        @ParameterizedTest
27        @MethodSource("failureCases")
28        void testUpdateAnnuncioStats_failure(Long idImmobile, String
29            tipoAggiornamento) throws GenericServiceException {
30            assertThrows(IllegalArgumentException.class, () ->
31                annuncioService.updateAnnuncioStats(idImmobile,
32                    tipoAggiornamento, TOKEN));
33
34            verify(annuncioService, never()).executeAndHandle(anyString(),
35                anyString(), any(), anyString(), any());
36        }
37    }
38
39    @Nested
40    @DisplayName("Casi di aggiornamento validi")
41    class UpdateStatsSuccessTests {
42
43        static Stream<Arguments> successCases() {
44            return Stream.of(
45                Arguments.of(ID_IMMOBILE_NOMINAL,
46                    TIPO_AGGIORNAMENTO_VISUALIZZAZIONE),
47                Arguments.of(ID_IMMOBILE_MINIMAL,
48                    TIPO_AGGIORNAMENTO_OFFERTA),
49                Arguments.of(ID_IMMOBILE_NOMINAL, TIPO_AGGIORNAMENTO_VISITA)
50            );
51        }
52
53        @ParameterizedTest
54        @MethodSource("successCases")
55        void testUpdateAnnuncioStats_success(Long idImmobile, String
56            tipoAggiornamento) throws GenericServiceException {
57            AnnuncioDTO fakeResponse = new AnnuncioDTO();
58
59            mockExecuteAndHandle(fakeResponse);
60        }
61    }
62 }
```

```

53     AnnuncioDTO result =
        annuncioService.updateAnnuncioStats(idImmobile,
            tipoAggiornamento, TOKEN);
54
55     assertNotNull(result);
56     assertEquals(fakeResponse, result);
57
58     verifySuccessCallOfExecuteAndHandle(idImmobile, tipoAggiornamento);
59 }
60
61
62 private void mockExecuteAndHandle(AnnuncioDTO fakeResponse) throws
    GenericServiceException {
63     doReturn(fakeResponse).when(annuncioService).executeAndHandle(
64         eq("/updateStats"),
65         eq("PUT"),
66         any(UpdateAnnuncioDTO.class),
67         eq(TOKEN),
68         eq(AnnuncioDTO.class)
69     );
70 }
71
72 private void verifySuccessCallOfExecuteAndHandle(Long idImmobile, String
    tipoAggiornamento) throws GenericServiceException {
73     verify(annuncioService).executeAndHandle(
74         eq("/updateStats"),
75         eq("PUT"),
76         argThat(updateDto ->
77             updateDto instanceof UpdateAnnuncioDTO
78                 && ((UpdateAnnuncioDTO) updateDto)
79                 .getIdImmobile().equals(idImmobile) &&
80                 ((UpdateAnnuncioDTO) updateDto)
81                 .getTipoAggiornamento().equals(tipoAggiornamento)
82         ),
83         eq(TOKEN),
84         eq(AnnuncioDTO.class)
85     );
86 }
87 }

```

Listing 1: Estratto codice di *AnnuncioServiceTest.java*

4.1.2 Metodo `registerAgenzia`

Tabella 3: Classi di Equivalenza per `registerAgenzia`

Attributo	Classe di Equivalenza	Valori di Test
nome	Valido (CE1: Stringa non vuota, no caratteri speciali)	"test agenzia 123", "a"
	Non valido (CE2: Stringa vuota)	""
	Non valido (CE3: null)	null
	Non valido (CE4: Contiene caratteri speciali)	"test agenzia 123!"
partitaIva	Valido (CE1: Solo numeri, 11 cifre)	"12345678901"
	Non valido (CE2: null)	null
	Non valido (CE3: Stringa vuota)	""
	Non valido (CE4: Caratteri alfabetici)	"1234567890a"
	Non valido (CE5: Caratteri speciali)	"1234567890!"
	Non valido (CE6: Non 11 cifre)	"1234567890", "123456789012"
indirizzo	Valido (CE1: Stringa non vuota)	"Via Roma 1", "V"
	Non valido (CE2: Stringa vuota)	""
	Non valido (CE3: null)	null
email	Valido (CE1: Email formattata correttamente)	"prova@prova.com"
	Non valido (CE2: null)	null
	Non valido (CE3: Stringa vuota)	""
	Non valido (CE4: Senza ".")	"prova@prova"
	Non valido (CE5: Senza "@")	"prova.com"
telefono	Valido (CE1: Solo cifre, 10 numeri)	"1234567890"
	Non valido (CE2: null)	null
	Non valido (CE3: Stringa vuota)	""
	Non valido (CE4: Caratteri alfabetici)	"123456789a"
	Non valido (CE5: Caratteri speciali)	"123456789!"
	Non valido (CE6: Non 10 cifre)	"123456789", "12345678901"
logo	Valido (CE1: Stringa non vuota)	"logo.png"
	Non valido (CE2: null)	null
	Non valido (CE3: Stringa vuota)	""
logoFile	Valido (CE1: Immagine < 2MB, formato 1:1, max 512x512)	File immagine con dimensioni 1x1, 512x512 e peso < 2MB
	Non valido (CE2: null)	null
	Non valido (CE3: Peso > 2MB)	File immagine con peso 2.1MB
	Non valido (CE4: Formato non 1:1)	File immagine con dimensioni 200x300

Continua nella pagina successiva

Attributo	Classe di Equivalenza	Valori di Test
password	Non valido (CE5: Dimensioni > 512x512)	File immagine con dimensioni 600x600
	Non valido (CE6: Non è un'immagine)	File di testo "file.txt"
	Valido (CE1: >= 8 caratteri, 1 maiuscola, 1 numero)	"Password1"
	Non valido (CE2: null)	null
	Non valido (CE3: Stringa vuota)	""
	Non valido (CE4: Troppo corta)	"Passwo7"
	Non valido (CE5: Senza maiuscola)	"password1"
	Non valido (CE6: Senza numero)	"Password"

Codice test

```
1 @ExtendWith(MockitoExtension.class)
2 public class AgenziaServiceTest {
3     @InjectMocks
4     @Spy
5     private AgenziaService agenziaService;
6
7     @Nested
8     @DisplayName("Casi di registrazione agenzia non validi")
9     class RegisterAgencyFailureTests {
10
11         static Stream<Arguments> failureCases() {
12             return Stream.of(
13                 Arguments.of(NOME_AGENZIA_EMPTY, PARTITA_IVA_AGENZIA,
14                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
15                     LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
16                 Arguments.of(NOME_AGENZIA_NULL, PARTITA_IVA_AGENZIA,
17                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
18                     LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
19                 Arguments.of(NOME_AGENZIA_WITH_SPECIAL_CHARACTER,
20                     PARTITA_IVA_AGENZIA, INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
21                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
22                     LOGO_FILE_WITH_NOMINAL_SIZE),
23                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA_EMPTY,
24                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
25                     LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
26                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA_NULL,
27                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
28                     LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
29                 Arguments.of(NOME_AGENZIA,
30                     PARTITA_IVA_AGENZIA_WITH_LETTERS, INDIRIZZO_AGENZIA,
31                     EMAIL_AGENZIA, TELEFONO_AGENZIA, LOGO_AGENZIA,
32                     PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
33                 Arguments.of(NOME_AGENZIA,
34                     PARTITA_IVA_AGENZIA_WITH_10_DIGITS, INDIRIZZO_AGENZIA,
35                     EMAIL_AGENZIA, TELEFONO_AGENZIA, LOGO_AGENZIA,
36                     PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
37                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
38                     INDIRIZZO_AGENZIA_EMPTY, EMAIL_AGENZIA,
39                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
40                     LOGO_FILE_WITH_NOMINAL_SIZE),
41                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
42                     INDIRIZZO_AGENZIA_NULL, EMAIL_AGENZIA,
43                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
44                     LOGO_FILE_WITH_NOMINAL_SIZE),
45                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
46                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA_EMPTY,
47                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
48                     LOGO_FILE_WITH_NOMINAL_SIZE),
49                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
50                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA_NULL,
51                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
52                     LOGO_FILE_WITH_NOMINAL_SIZE),
53                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
54                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA_WITHOUT_AT,
55                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
56                     LOGO_FILE_WITH_NOMINAL_SIZE),
57                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
58                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA_WITHOUT_DOT,
59                     TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
60                     LOGO_FILE_WITH_NOMINAL_SIZE),
61                 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
62                     INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
```



```

TELEFONO_AGENZIA_EMPTY, LOGO_AGENZIA, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
27 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
TELEFONO_AGENZIA_NULL, LOGO_AGENZIA, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
28 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
TELEFONO_AGENZIA_WITH_LETTERS, LOGO_AGENZIA, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
29 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
TELEFONO_AGENZIA_WITH_9_DIGITS, LOGO_AGENZIA, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
30 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA,
TELEFONO_AGENZIA_WITH_SPECIAL_CHARACTER, LOGO_AGENZIA,
PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
31 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA_EMPTY, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
32 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA_NULL, PASSWORD,
LOGO_FILE_WITH_NOMINAL_SIZE),
33 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD_EMPTY,
LOGO_FILE_WITH_NOMINAL_SIZE),
34 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD_NULL,
LOGO_FILE_WITH_NOMINAL_SIZE),
35 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD_WITHOUT_UPPERCASE,
LOGO_FILE_WITH_NOMINAL_SIZE),
36 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD_WITHOUT_NUMBER,
LOGO_FILE_WITH_NOMINAL_SIZE),
37 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD_WITH_7_DIGITS,
LOGO_FILE_WITH_NOMINAL_SIZE),
38 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD, LOGO_FILE_NULL),
39 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD, LOGO_FILE_NOT_IMAGE),
40 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD, LOGO_FILE_NOT_SQUARE),
41 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD, LOGO_FILE_TOO_LARGE_FILE_SIZE),
42 Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
LOGO_AGENZIA, PASSWORD, LOGO_FILE_TOO_LARGE_IMAGE_SIZE)
43
44 );

```

```

45     }
46
47     @ParameterizedTest
48     @MethodSource("failureCases")
49     void testRegisterAgency_failure(String nome, String partitaIva, String
        indirizzo, String email, String telefono, String logo, String
        password, File logoFile) throws GenericServiceException {
50         AgenziaImmobiliare agenzia = new AgenziaImmobiliare(nome,
            partitaIva, indirizzo, email, telefono, logo);
51
52         assertThrows(GenericServiceException.class, () ->
53             agenziaService.registerAgenzia(agenzia, logoFile,
54                 password));
55
56         verify(agenziaService,
57             never()).executeAndHandleMultipart(anyString(), anyString(),
58                 any(), anyString(), anyString(), any());
59     }
60 }
61
62 @Nested
63 @DisplayName("Casi di aggiornamento validi")
64 class RegisterAgencySuccessTests {
65
66     static Stream<Arguments> successCases() {
67         return Stream.of(
68             Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
69                 INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
70                 LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_NOMINAL_SIZE),
71             Arguments.of(NOME_AGENZIA_MINIMAL, PARTITA_IVA_AGENZIA,
72                 INDIRIZZO_AGENZIA_MINIMAL, EMAIL_AGENZIA,
73                 TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
74                 LOGO_FILE_WITH_MINIMAL_SIZE),
75             Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
76                 INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
77                 LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_MAX_SIZE),
78             Arguments.of(NOME_AGENZIA, PARTITA_IVA_AGENZIA,
79                 INDIRIZZO_AGENZIA, EMAIL_AGENZIA, TELEFONO_AGENZIA,
80                 LOGO_AGENZIA, PASSWORD, LOGO_FILE_WITH_MAX_FILE_SIZE),
81             Arguments.of(NOME_AGENZIA_MINIMAL, PARTITA_IVA_AGENZIA,
82                 INDIRIZZO_AGENZIA_MINIMAL, EMAIL_AGENZIA,
83                 TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
84                 LOGO_FILE_WITH_MINIMAL_FILE_SIZE),
85             Arguments.of(NOME_AGENZIA_MINIMAL, PARTITA_IVA_AGENZIA,
86                 INDIRIZZO_AGENZIA_MINIMAL, EMAIL_AGENZIA,
87                 TELEFONO_AGENZIA, LOGO_AGENZIA, PASSWORD,
88                 LOGO_FILE_WITH_NOMINAL_FILE_SIZE)
89         );
90     }
91
92     @ParameterizedTest
93     @MethodSource("successCases")
94     void testRegisterAgency_success(String nome, String partitaIva, String
        indirizzo, String email, String telefono, String logo, String
        password, File logoFile) throws GenericServiceException {
95         mockExecuteAndHandleMultipart();
96         AgenziaImmobiliare agenzia = new AgenziaImmobiliare(nome,
97             partitaIva, indirizzo, email, telefono, logo);
98         agenziaService.registerAgenzia(agenzia, logoFile, password);
99         verifySuccessCallOfExecuteAndHandleMultipart(agenzia, logoFile,
100             password);
101     }
102 }

```

```

83     private void mockExecuteAndHandleMultipart() throws
      GenericServiceException {
84         doReturn(null).when(agenziaService).executeAndHandleMultipart(
85             eq("/register"),
86             eq("POST"),
87             any(byte[].class),
88             anyString(),
89             eq(null),
90             eq(null)
91         );
92     }
93
94     private void
      verifySuccessCallOfExecuteAndHandleMultipart(AgenziaImmobiliare
      agenzia, File logoFile, String password) throws
      GenericServiceException {
95         ArgumentCaptor<byte[]> bodyCaptor =
          ArgumentCaptor.forClass(byte[].class);
96         ArgumentCaptor<String> contentTypeCaptor =
          ArgumentCaptor.forClass(String.class);
97
98         verify(agenziaService, times(1)).executeAndHandleMultipart(
99             eq("/register"),
100             eq("POST"),
101             bodyCaptor.capture(),
102             contentTypeCaptor.capture(),
103             eq(null),
104             eq(null)
105         );
106
107         byte[] capturedBody = bodyCaptor.getValue();
108         String capturedContentType = contentTypeCaptor.getValue();
109
110         String bodyAsString = new String(capturedBody);
111
112         assertTrue(bodyAsString.contains("name=\"nome\""));
113         assertTrue(bodyAsString.contains(agenzia.getNome()));
114
115         assertTrue(bodyAsString.contains("name=\"partitaIva\""));
116         assertTrue(bodyAsString.contains(agenzia.getPartitaIva()));
117
118         assertTrue(bodyAsString.contains("name=\"indirizzo\""));
119         assertTrue(bodyAsString.contains(agenzia.getIndirizzo()));
120
121         assertTrue(bodyAsString.contains("name=\"email\""));
122         assertTrue(bodyAsString.contains(agenzia.getEmail()));
123
124         assertTrue(bodyAsString.contains("name=\"telefono\""));
125         assertTrue(bodyAsString.contains(agenzia.getTelefono()));
126
127         assertTrue(bodyAsString.contains("name=\"password\""));
128         assertTrue(bodyAsString.contains(password));
129
130         assertNotNull(capturedContentType);
131         assertTrue(capturedContentType.startsWith("multipart/form-data;
          boundary="));
132
133         assertTrue(bodyAsString.contains("name=\"logo\""));
134         assertTrue(bodyAsString.contains(logoFile.getName()));
135     }
136 }
137

```

Listing 2: Estratto codice di *AgenziaServiceTest.java*

4.1.3 Descrizione dei test effettuati

Sono stati sviluppati test automatici per verificare il corretto funzionamento di alcuni servizi offerti dalle classi `AnnuncioService` e `AgenziaService`. I test sono stati realizzati utilizzando **JUnit 5** in combinazione con **Mockito**, che ha permesso di isolare il comportamento delle classi testate e di gestire in maniera flessibile le dipendenze.

4.1.4 Test su `AnnuncioService`

I test relativi alla classe `AnnuncioService` si concentrano sulla validazione dell'aggiornamento delle statistiche di un annuncio immobiliare. I test sono suddivisi in due categorie principali:

- **Casi di aggiornamento validi:** verificano che l'aggiornamento delle statistiche avvenga correttamente quando i parametri in ingresso sono validi.
- **Casi di aggiornamento non validi:** controllano che il servizio generi un'eccezione quando vengono forniti parametri errati.

4.1.5 Test su `AgenziaService`

Per la classe `AgenziaService` sono stati implementati test per verificare la registrazione di una nuova agenzia immobiliare. Anche in questo caso i test sono stati suddivisi in due categorie principali:

- **Casi di registrazione validi:** verificano che la registrazione dell'agenzia avvenga correttamente quando tutti i parametri forniti sono validi.
- **Casi di registrazione non validi:** controllano che il servizio generi un'eccezione quando vengono forniti parametri errati.

4.1.6 Utilizzo di Mockito

Mockito è stato impiegato per creare oggetti fittizi (*mock* e *spy*) e simulare il comportamento di metodi e componenti esterni. In particolare:

- L'annotazione `@InjectMocks` insieme a `@Spy` è stata utilizzata per creare istanze parzialmente reali delle classi testate, consentendo di sostituire solo alcuni metodi con implementazioni predefinite utili per il test.
- Sono state utilizzate le funzioni di stub di Mockito, come `doReturn(...).when(...)` per simulare risposte specifiche e controllare il flusso di esecuzione in base agli input forniti.
- Mediante le funzioni di verifica (`verify`, `verify(never)`), è stato possibile accertarsi che le interazioni con i metodi dipendenti avvenissero correttamente oppure venissero evitati in presenza di dati non validi.
- L'uso di `ArgumentCaptor` ha permesso di catturare e ispezionare i parametri passati ai metodi mockati, garantendo che i dati inviati nelle richieste corrispondessero alle aspettative.

Questa strategia ha garantito che il test rimanesse isolato, focalizzandosi esclusivamente sulla logica della classe in esame, senza dipendere dall'effettiva implementazione dei servizi esterni.

4.1.7 Utilizzo dei Test Parametrizzati

Per aumentare la copertura dei casi di test e rendere il codice più manutenibile, sono stati adottati i test parametrizzati tramite le annotazioni `@ParameterizedTest` e `@MethodSource`:

- **Test parametrizzati:** Permettono di eseguire lo stesso metodo di test con differenti insiemi di dati. Questo approccio consente di ridurre la duplicazione del codice, verificando in maniera sistematica tutte le combinazioni possibili di input validi e non validi.
- **MethodSource:** Viene utilizzato per fornire uno stream di argomenti al test. In questo modo, è possibile definire scenari di test complessi in maniera centralizzata e leggibile.
- La struttura parametrizzata consente di documentare chiaramente le condizioni di errore e i casi in cui il sistema deve rispondere con un'eccezione oppure procedere correttamente.

4.2 Valutazione dell'usabilità

4.2.1 Expert review

La valutazione dell'usabilità è stata condotta da degli "esperti dell'usabilità", ossia:

- Da noi stessi.
- Da alcuni nostri colleghi universitari iscritti al terzo anno della facoltà di informatica.

Di seguito verranno elencati i vari punti della checklist per l'Expert Review.

Tabella 4: Checklist per per l'Expert Review

Area di Valutazione	Elemento	Dettagli
Interfaccia e Navigazione	Chiarezza del Layout e Organizzazione	Layout intuitivo e coerente in tutte le schermate; gerarchia visiva chiara (titoli, pulsanti, form, ecc.).
Interfaccia e Navigazione	Coerenza Grafica e Stile	Uniformità di colori, tipografia e icone; uso costante dei termini ed etichette (label, messaggi dei popup).
Interfaccia e Navigazione	Facilità di Navigazione	Navigazione semplice e percorsi logici tra le funzionalità; presenza di elementi (pulsanti "indietro") facilmente individuabili.
Validazione degli Input e Gestione degli Errori	Verifica dei Dati Inseriti	Controllo dei campi obbligatori (es. nome, email, partita IVA, ecc.) e validazione in tempo reale degli input errati o mancanti.
Validazione degli Input e Gestione degli Errori	Gestione dei Casi Limite e Inconsistenze	Rifiuto di dati non validi (numeri negativi, stringhe vuote, formati errati); o valori anomali.
Feedback e Comunicazione all'Utente	Feedback di Errore	Messaggi di errore chiari e specifici
Feedback e Comunicazione all'Utente	Conferme di Successo	Notifiche visive o messaggi di conferma al completamento delle operazioni (registrazione, aggiornamento, ecc...).
Performance e Reattività	Tempi di Risposta Accettabili	Velocità nell'elaborazione delle operazioni e indicazioni di caricamento quando richiesto.

4.2.2 Inspections

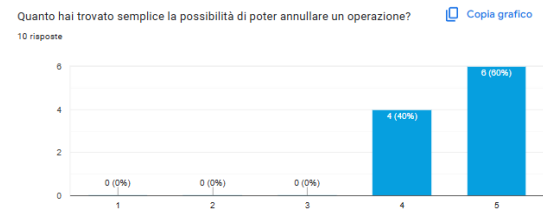
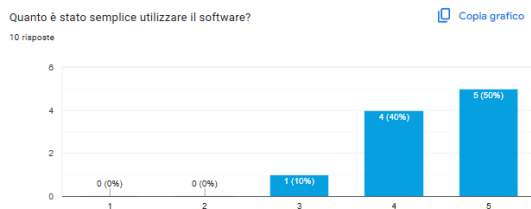
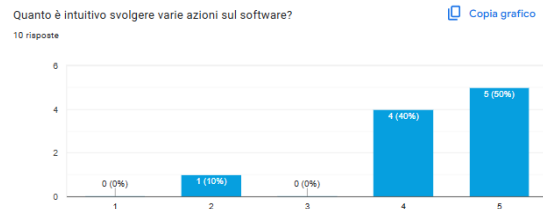
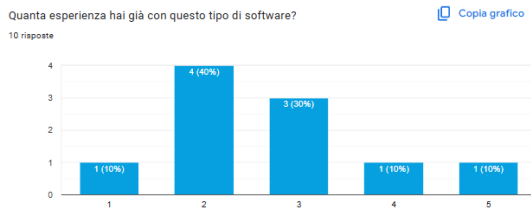
Attraverso l'osservazione durante l'utilizzo del software da parte degli utenti scelti, sono stati trovati alcuni difetti:

- Mancanza del **caricamento** durante alcune operazioni che richiedono molto tempo.
- Errori nell'**aspetto estetico** introdotti per distrazione.
- Cambiato il nome di alcuni **bottoni** per annullare operazioni in modo da renderli più specifici.

Tutti gli errori appena elencati, ovviamente, sono stati risolti in seguito alla loro scoperta.

4.2.3 Risultati del form

Tramite l'utilizzo di **Google Form** è stato condotto un sondaggio per gli utenti che hanno usato l'applicazione, subito dopo averla usata. Questo ci ha permesso di capire, oltre all'ispezione visiva, i problemi riscontrati da parte di un utente, relazionandoli con dati come l'età, il titolo di studio e l'esperienza con l'utilizzo di software di questa tipologia. Di seguito i risultati delle risposte del form.



Da come possiamo osservare le persone che **NON** hanno **esperienza** con l'uso di questa tipologia di software hanno riscontrato leggeri problemi nello svolgere alcune azioni. Al contrario, coloro che hanno già avuto a che fare con software simili non hanno avuto alcun tipo di problema, riscontrando un'esperienza piacevole. Questo indica un'alta **consistenza esterna**, rendendo il procedimento delle funzioni abituali simile a quello di altri software della stessa categoria. Tra la lista di questi software usati troviamo *Booking*, *Immobiliare* e *Idealista*.

Data di nascita

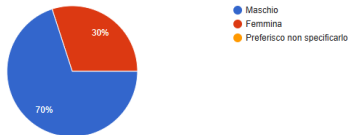
10 risposte

ott 1959	31
set 1965	17
nov 1977	11
mar 1982	16
mag 2003	16
giu 2003	11
lug 2003	31
ott 2003	30
mar 2004	29
giu 2005	30

L'**età** delle persone a cui è stato sottoposto il sondaggio varia tra i 19 fino a i 65 anni, in modo da avere un vasto feedback, tramite l'utilizzo del software da persone abituate *diversamente* con la tecnologia moderna. Questa differenza però non è stata notata nell'utilizzo, confermando che il fattore principale per avere un'esperienza semplice e piacevole rimane di conoscere già software simili.

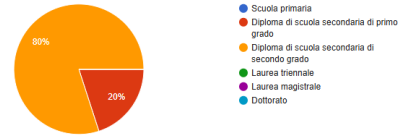
Genere
10 risposte

[Copia grafico](#)



Qual è il tuo titolo di studio
10 risposte

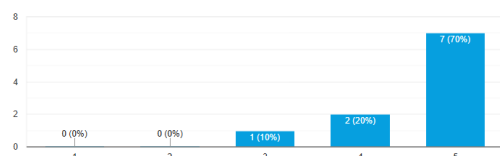
[Copia grafico](#)



Anche il **genere** e il **titolo di studio** non hanno riportato alcuna difficoltà nel utilizzo quotidiano.

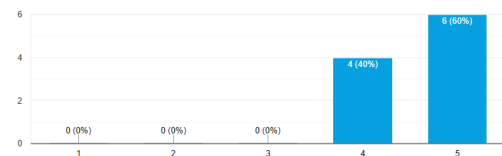
Quanto è stato soddisfacente utilizzare il software?
10 risposte

[Copia grafico](#)



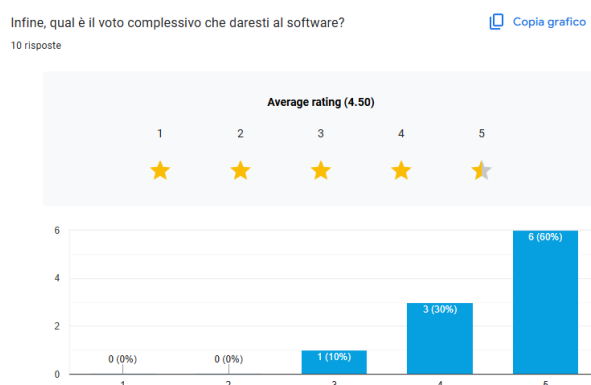
Quanto hai trovato adatto lo stile adottato alla tipologia del software?
10 risposte

[Copia grafico](#)



La **parte estetica** è risultata per la maggior parte degli utenti semplice e soprattutto intuitiva. A detta di alcuni *"i colori trasmettono serenità e tranquillità"*, rendendolo soddisfacente nell'utilizzo quotidiano.

Il **voto finale** assegnatoci dagli utenti è riportato nell'immagine seguente.



4.2.4 Valutazione interna delle risposte

Per ogni domanda sono stati assegnati dei pesi rispettivi in base all'importanza. In questo modo possiamo calcolare una media ponderata. I pesi scelti sono i seguenti:

- Semplicità d'uso (0.2)
- Facilità di annullare le operazioni (0,1)
- Stile appropriato (0.15)
- Intuitività dell'uso (0.5)
- Soddisfazione nell'utilizzo (0.3)

Inoltre questi valori sono stati combinati con un peso dato all'esperienza nell'uso di software di questa tipologia.

- Voto ≤ 2 (1.3)
- $2 > \text{Voto} \leq 4$ (1)
- Voto = 5 (0.8)

Infine è stata calcolata la media ponderata di tutti i voti dati alle risposte relative alla qualità del software moltiplicando il voto ricavato da ogni utente per la sua esperienza pesata. Ecco il risultato finale ottenuto:

Voto ponderato	Media ponderata con esperienze
5,85	4,442035398
4,85	
4,6	
5,98	
4	
5,395	
3,9	
6,37	
4,7	
4,55	